

# Graphisme dans le jeu vidéo

## Table des matières

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| Session 1 : Géométrie et Découverte de l'API .....                         | 3  |
| Présentation du cours .....                                                | 3  |
| Découverte de l'environnement : Babylon.js et le Playground .....          | 4  |
| Rappels mathématiques : Vecteurs et Matrices .....                         | 4  |
| La géométrie bas niveau .....                                              | 5  |
| Atelier pratique : Création d'un mesh à la main .....                      | 6  |
| Démonstration : Construire un maillage de A à Z .....                      | 8  |
| Session 2 : Transformations et Espaces de Coordonnées .....                | 11 |
| Le voyage d'un Vertex : Le Pipeline de Transformation .....                | 11 |
| Comprendre les Matrices : L'approche « Calculatrice » .....                | 14 |
| La Matrice Identité : La machine au repos .....                            | 14 |
| La Matrice de Scale : Le multiplicateur .....                              | 14 |
| Le laboratoire de test : L'ordre des opérations .....                      | 15 |
| Anatomie des matrices de transformation .....                              | 15 |
| La Matrice de Translation et l'astuce de la coordonnée W .....             | 16 |
| Atelier Desmos : Visualiser les matrices en 2D .....                       | 17 |
| Le lien avec la géométrie : La boucle est bouclée .....                    | 18 |
| Hiérarchie de scène : Le Parenting .....                                   | 19 |
| Atelier : Le Train et le Passager .....                                    | 19 |
| Concepts clés à retenir (Les pièges classiques) .....                      | 20 |
| Session 3 : Création de Shaders dans Babylon.js (Partie 1) .....           | 22 |
| De la copie manuelle à la presse d'imprimerie .....                        | 22 |
| Qu'est-ce qu'un Fragment Shader ? .....                                    | 22 |
| Pourquoi les shaders sont-ils rapides ? .....                              | 22 |
| Pourquoi les shaders font peur ? .....                                     | 23 |
| Le Pipeline de Rendu Programmable .....                                    | 23 |
| GLSL : Le langage des Shaders .....                                        | 23 |
| Shaders dans Babylon.js : ShaderMaterial .....                             | 24 |
| Uniforms et Varyings .....                                                 | 25 |
| Session 4 : Création de Shaders dans Babylon.js (Partie 2) .....           | 26 |
| La fonction step .....                                                     | 26 |
| La terre vue par NASA .....                                                | 26 |
| Session 5 : Lumière et matériaux .....                                     | 27 |
| Partie 1 : L'Éclairage Mathématique — La Loi de Lambert .....              | 27 |
| Partie 2 : Sources de Lumières et Spéculaire .....                         | 32 |
| Concepts clés à retenir .....                                              | 39 |
| Session 6 : L'Éclairage Statique .....                                     | 40 |
| Partie 1 : Le Problème — L'Éclairage Temps Réel est Trop Cher .....        | 40 |
| Partie 2 : Les Lightmaps — L'Éclairage Cuit au Four .....                  | 40 |
| Partie 3 : L'Ambient Occlusion (AO) .....                                  | 43 |
| Partie 4 : Les Light Probes — L'Éclairage pour les Objets Dynamiques ..... | 45 |
| Partie 5 : L'Éclairage Mixte (Mixed Lighting) .....                        | 47 |

|                                                                      |    |
|----------------------------------------------------------------------|----|
| Partie 6 : Reflection Probes — Les Réflexions Baked .....            | 47 |
| Partie 7 : Le Pipeline Complet d'Éclairage Statique .....            | 48 |
| Partie 8 : Le Texture Baking pour Personnages .....                  | 48 |
| Concepts clés à retenir .....                                        | 50 |
| Travail Pratique : Baked Lighting en Three.js .....                  | 51 |
| Session 7 : Création de Post-Process dans Babylon.js .....           | 54 |
| Qu'est-ce qu'un Post-Process ? .....                                 | 54 |
| Post-Process Pipeline dans Babylon.js .....                          | 54 |
| Post-Process Intégrés .....                                          | 54 |
| Post-Process Custom .....                                            | 54 |
| HDR et Tone Mapping .....                                            | 55 |
| Session 8 : Rendering Pipeline (Partie 1) .....                      | 56 |
| Vue d'Ensemble du Pipeline .....                                     | 56 |
| Vertex Shader en Détail .....                                        | 56 |
| Rasterization .....                                                  | 56 |
| Fragment Shader en Détail .....                                      | 56 |
| Tests et Blending .....                                              | 57 |
| Session 9 : Rendering Pipeline (Partie 2) .....                      | 58 |
| Forward vs Deferred Rendering .....                                  | 58 |
| G-Buffer .....                                                       | 58 |
| Culling .....                                                        | 58 |
| Draw Calls et Batching .....                                         | 58 |
| Babylon.js et le Pipeline .....                                      | 59 |
| Session 10 : Techniques d'Optimisation et Profilage (Partie 1) ..... | 60 |
| Pourquoi Optimiser ? .....                                           | 60 |
| Outils de Profilage .....                                            | 60 |
| Métriques Clés .....                                                 | 60 |
| Optimisation Géométrique .....                                       | 60 |
| Optimisation des Draw Calls .....                                    | 61 |
| Session 11 : Techniques d'Optimisation et Profilage (Partie 2) ..... | 62 |
| Optimisation des Textures .....                                      | 62 |
| Optimisation des Shaders .....                                       | 62 |
| Level of Detail (LOD) dans Babylon.js .....                          | 62 |
| Gestion de la Mémoire GPU .....                                      | 62 |
| WebGPU vs WebGL .....                                                | 63 |
| Session 12 : Exploration d'une API de Programmation Graphique .....  | 64 |
| APIs Graphiques : Vue d'Ensemble .....                               | 64 |
| WebGPU : La Nouvelle API Web .....                                   | 64 |
| Comparaison WebGL vs WebGPU .....                                    | 64 |
| WebGPU dans Babylon.js .....                                         | 64 |
| WGSL : Le Langage de Shader WebGPU .....                             | 65 |
| Conclusion du Cours .....                                            | 65 |

# Session 1 : Géométrie et Découverte de l'API

## 🔗 Objectifs de la session

Présenter le cours et son fonctionnement, découvrir l'écosystème Babylon.js, rappeler les notions mathématiques essentielles, et construire un maillage 3D à la main pour comprendre la structure intime d'un modèle.

## Présentation du cours

### Orientations et évaluation

Ce cours explore la programmation graphique 3D temps réel appliquée au jeu vidéo, en utilisant entre autres Babylon.js comme moteur, mais aussi d'autres outils et frameworks (Unity, Unreal Engine, Blender, Godot).

### Évaluation :

- Travaux pratiques (TP) : Mise en pratique des concepts de chaque session.
- Projet final : Création d'un projet utilisant l'ensemble des principes abordés.

**Approche pédagogique :** Chaque session combine théorie et atelier pratique. Le code est écrit en JavaScript/TypeScript et s'exécute directement dans le navigateur. Des démonstrations seront aussi faites sur les engins commerciaux, pour montrer les analogies, la programmation graphique étant indépendante du moteur utilisé.

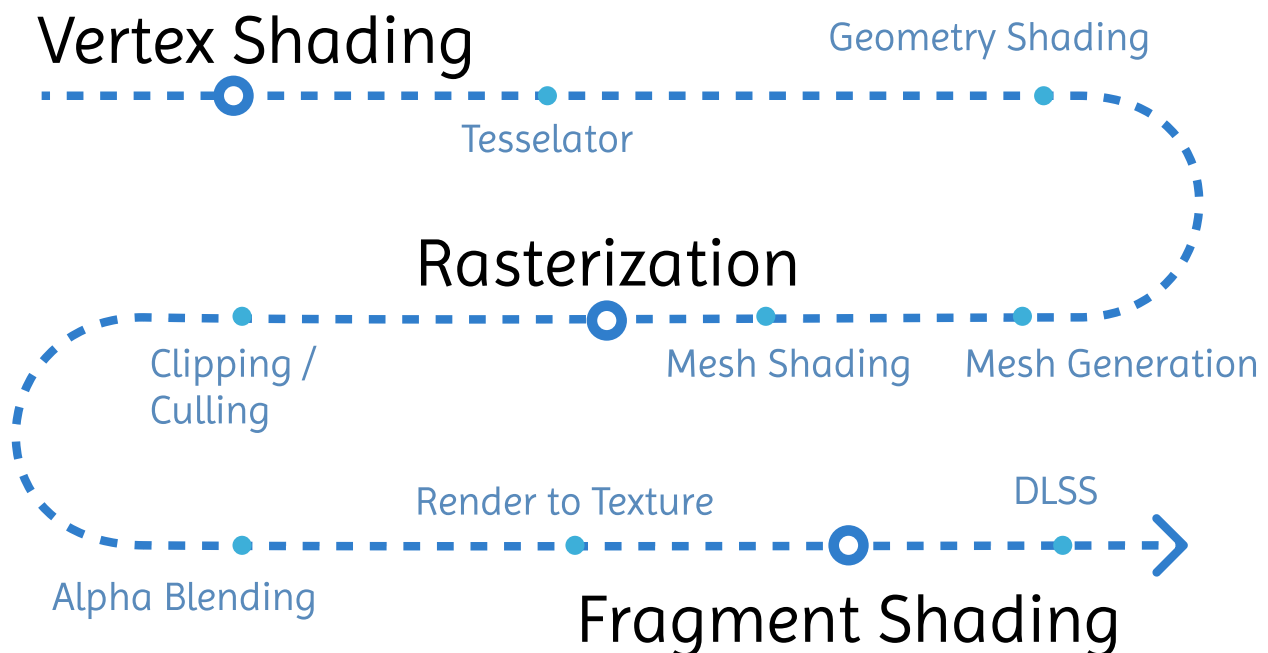


Fig. 1. – Notre parcours dans l'univers du GPU.

## Introduction à la programmation graphique

La programmation graphique consiste à décrire au GPU (carte graphique) comment transformer des données mathématiques (points, vecteurs, matrices) en pixels colorés à l'écran. C'est la rencontre entre les mathématiques, la physique (optique) et l'informatique.

Contrairement à la programmation classique (CPU, séquentielle), la programmation graphique exploite le parallélisme massif du GPU : des milliers de calculs exécutés simultanément.

## Découverte de l'environnement : Babylon.js et le Playground

### L'écosystème Babylon.js

Babylon.js est un moteur 3D open-source écrit en TypeScript, s'exécutant dans le navigateur via WebGL et WebGPU. Il offre :

- Un moteur de rendu temps réel (PBR, lumières, ombres, post-process).
- Un éditeur visuel de shaders (Node Material Editor).
- Une API complète pour la création de scènes, d'animations et d'interactions.

**Avantage clé :** Aucune installation requise. Tout s'exécute dans le navigateur, et il est possible de partager facilement les exemples via URL.

### 🔗 Le Babylon.js Playground

Le **Playground** (<https://playground.babylonjs.com>) est l'outil idéal pour prototyper :

- Zéro installation : on écrit du code JS directement dans le navigateur.
- Itération rapide : modification en direct, rendu instantané.
- Partage : chaque scène a une URL unique partageable.
- Modèles : des centaines d'exemples officiels pour apprendre.

C'est l'environnement que nous utiliserons pour les ateliers pratiques.

## Rappels mathématiques : Vecteurs et Matrices

### Vecteurs (Vector3)

Un vecteur représente une direction et une magnitude dans l'espace 3D. Dans Babylon.js, on utilise la classe `BABYLON.Vector3`.

#### Opérations essentielles :

```
const a = new BABYLON.Vector3(1, 2, 3);
const b = new BABYLON.Vector3(4, 5, 6);

a.add(b);           // Addition
a.subtract(b);      // Soustraction
a.scale(2);         // Multiplication par scalaire
BABYLON.Vector3.Dot(a, b); // Produit scalaire
BABYLON.Vector3.Cross(a, b); // Produit vectoriel
a.normalize();       // Vecteur unitaire
a.length();         // Magnitude (norme)
```

- **Produit scalaire** : Mesure l'alignement entre deux vecteurs. Utilisé pour l'éclairage (angle entre normale et lumière).
- **Produit vectoriel** : Produit un vecteur perpendiculaire aux deux autres. Utilisé pour calculer les normales de surface.

## Matrices (Matrix)

Une matrice 4x4 représente une **transformation** dans l'espace (translation, rotation, scale, ou combinaison). Babylon.js utilise `BABYLON.Matrix`.

### Transformations courantes :

```
// Translation
const T = BABYLON.Matrix.Translation(1, 0, 0);

// Rotation (autour de Y)
const R = BABYLON.Matrix.RotationY(Math.PI / 4);

// Scale
const S = BABYLON.Matrix.Scaling(2, 2, 2);

// Composition : T * R * S (l'ordre compte !)
const world = T.multiply(R).multiply(S);
```

**Pourquoi 4x4 ?** La 4ème coordonnée ( $w$ ) permet de unifier translations et rotations dans une seule matrice. Un point  $(x, y, z)$  devient  $(x, y, z, 1)$  en coordonnées homogènes.

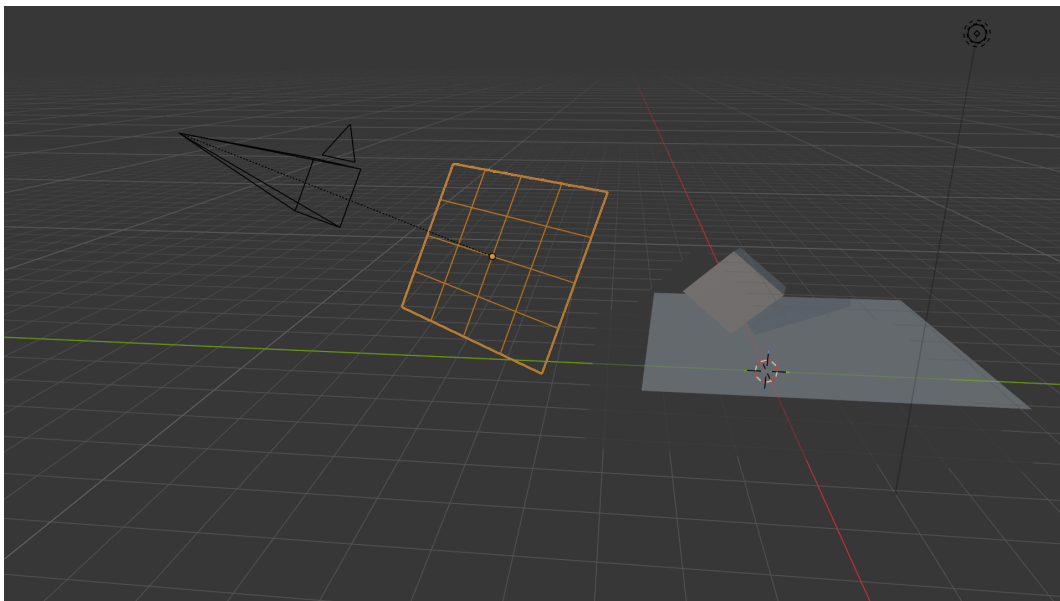


Fig. 2. – L'écran est une projection du monde en 3 dimensions.

## La géométrie bas niveau

### Composantes d'un Vertex

Un **vertex** (sommet) n'est pas qu'un point dans l'espace. Il peut contenir plusieurs attributs :

- **Position** : Coordonnées  $(x, y, z)$  dans l'espace objet. Obligatoire.
- **Normale** : Vecteur perpendiculaire à la surface, utilisé pour l'éclairage. Direction  $(n_x, n_y, n_z)$ .
- **UV** : Coordonnées de texture  $(u, v)$  dans  $[0, 1]$ , reliant le sommet à un pixel de la texture 2D.
- **Tangente** : Vecteur tangent à la surface, utilisé pour le normal mapping (espace tangent).
- **Couleur** : Couleur RGBA par sommet  $(r, g, b, a)$ , utilisée pour le vertex coloring.

Ces attributs sont stockés dans des **Vertex Buffers** séparés et envoyés au GPU.

## Types de primitives

En graphisme temps réel, toute surface est décomposée en triangles. Les types de primitives GPU sont :

- **Points** : Un point par vertex (rarement utilisé).
- **Lignes** : Une ligne par paire de vertices.
- **Line Strip** : Lignes connectées en chaîne.
- **Triangles** : Un triangle par triplet de vertices. C'est le type dominant.
- **Triangle Strip** : Triangles connectés partageant des arêtes (plus économe en vertices).
- **Triangle Fan** : Triangles partageant un vertex central.

Le triangle est la primitive de base car :

- Trois points définissent toujours un plan (pas d'ambiguïté).
- L'interpolation des attributs (UV, normales) est bien définie à l'intérieur.

## Atelier pratique : Création d'un mesh à la main

### ! Objectif de l'atelier

Construire un maillage triangulaire en manipulant directement les **Vertex Buffers** et l'**Index Buffer** via l'objet `VertexData` de Babylon.js. Comprendre la structure intime d'un modèle 3D.

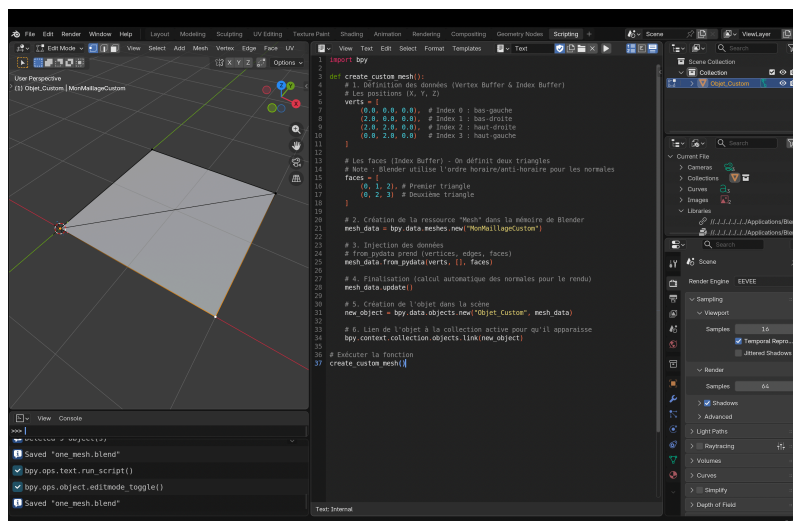


Fig. 3. – Le même maillage peut être créé visuellement dans Blender — les concepts de vertices, normales et UVs sont universels, indépendants du moteur utilisé.

## VertexData : L'objet de construction

BABYLON.VertexData est le conteneur bas niveau pour les données géométriques. Il permet de définir manuellement chaque buffer avant de les appliquer à un mesh.

### Buffers principaux :

- positions : Tableau de floats ( $x_0, y_0, z_0, x_1, y_1, z_1, \dots$ )
- normals : Tableau de floats ( $nx_0, ny_0, nz_0, \dots$ )
- uvs : Tableau de floats ( $u_0, v_0, u_1, v_1, \dots$ )
- indices : **Index Buffer** — tableau d'entiers désignant quels vertices forment chaque triangle.

*Créer un triangle à la main.*

```
// 1. Créer un mesh vide
const customMesh = new BABYLON.Mesh("custom", scene);

// 2. Définir les Vertex Buffers
const positions = [
    -1, -1, 0, // Vertex 0 : bas-gauche
    1, -1, 0, // Vertex 1 : bas-droite
    0, 1, 0 // Vertex 2 : haut-centre
];

const normals = [
    0, 0, 1, // Normale du Vertex 0 (vers la caméra)
    0, 0, 1, // Normale du Vertex 1
    0, 0, 1 // Normale du Vertex 2
];

const uvs = [
    0, 0, // UV du Vertex 0
    1, 0, // UV du Vertex 1
    0.5, 1 // UV du Vertex 2
];

// 3. Définir l'Index Buffer (quels vertices forment les triangles)
const indices = [0, 1, 2]; // Un seul triangle

// 4. Assembler le VertexData
const vertexData = new BABYLON.VertexData();
vertexData.positions = positions;
vertexData.normals = normals;
vertexData.uvs = uvs;
vertexData.indices = indices;

// 5. Appliquer au mesh
vertexData.applyToMesh(customMesh);

// 6. Assigner un matériau
const mat = new BABYLON.StandardMaterial("mat", scene);
mat.diffuseColor = new BABYLON.Color3(1, 0, 0);
mat.wireframe = false;
customMesh.material = mat;
```

## L'Index Buffer : Pourquoi et Comment ?

L'**Index Buffer** (ou tableau d'indices) indique au GPU dans quel ordre lire les vertices pour former des triangles.

### Pourquoi des indices ?

- Un vertex peut être partagé par plusieurs triangles (ex: un cube a 8 sommets mais 12 triangles).
- Sans index buffer, on dupliquerait les vertices partagés (gaspillage mémoire).
- Avec index buffer : 8 vertices + 36 indices au lieu de 36 vertices.

### Convention de winding (sens d'enroulement) :

- Babylon.js utilise le **counter-clockwise** (sens inverse horaire) vu de l'extérieur.
- indices = [0, 1, 2] : le triangle est visible depuis +Z.
- indices = [0, 2, 1] : le triangle est visible depuis -Z (backface culling).

## Démonstration : Construire un maillage de A à Z

Script complet pour le Babylon.js Playground. Ce code peut être collé directement dans le Playground. Il illustre la création d'un triangle et d'un carré (Quad) en manipulant les buffers.

```
export const createScene = function () {
    // 1. Initialisation de la scène basique
    const scene = new BABYLON.Scene(engine);

    // Caméra orbitale pour tourner autour de nos créations
    const camera = new BABYLON.ArcRotateCamera("camera", -Math.PI / 2, Math.PI / 2.5, 5,
BABYLON.Vector3.Zero(), scene);
    camera.attachControl(canvas, true);

    // Lumière basique
    const light = new BABYLON.HemisphericLight("light", new BABYLON.Vector3(0, 1, 0), scene);
    light.intensity = 0.7;

    // =====
    // EXEMPLE 1 : LE TRIANGLE
    // =====
    const triangleMesh = new BABYLON.Mesh("monTriangle", scene);

    const positionsTri = [
        -1, -1, 0,    // 0: bas-gauche
        1, -1, 0,     // 1: bas-droite
        0, 1, 0       // 2: haut-centre
    ];

    // Dans Babylon (Left-Handed), normales vers -1 en Z pour faire face à la caméra
    const normalsTri = [
        0, 0, -1,
        0, 0, -1,
        0, 0, -1
    ];

    const uvsTri = [
        0, 0,
        1, 0,
```

```

        0.5, 1
    ];

    const indicesTri = [0, 1, 2];

    const vertexDataTri = new BABYLON.VertexData();
    vertexDataTri.positions = positionsTri;
    vertexDataTri.normals = normalsTri;
    vertexDataTri.uvs = uvsTri;
    vertexDataTri.indices = indicesTri;

    vertexDataTri.applyToMesh(triangleMesh);

    // Matériau rouge pour le triangle
    const matTri = new BABYLON.StandardMaterial("matTri", scene);
    matTri.diffuseColor = new BABYLON.Color3(1, 0, 0);
    // matTri.wireframe = true; // Décommenter pour tester
    triangleMesh.material = matTri;

    triangleMesh.position.x = -1.5;

    // =====
    // EXEMPLE 2 : LE CARRÉ (QUAD) AVEC TEXTURE
    // =====
    const quadMesh = new BABYLON.Mesh("monCarre", scene);

    const positionsQuad = [
        -1, -1, 0, // 0 : bas-gauche
        1, -1, 0, // 1 : bas-droite
        1, 1, 0, // 2 : haut-droite
        -1, 1, 0 // 3 : haut-gauche
    ];

    const normalsQuad = [
        0, 0, -1,
        0, 0, -1,
        0, 0, -1,
        0, 0, -1
    ];

    const uvsQuad = [
        0, 0, // 0
        1, 0, // 1
        1, 1, // 2
        0, 1 // 3
    ];

    const indicesQuad = [0, 1, 2, 0, 2, 3];

    const vertexDataQuad = new BABYLON.VertexData();
    vertexDataQuad.positions = positionsQuad;
    vertexDataQuad.normals = normalsQuad;
    vertexDataQuad.uvs = uvsQuad;
    vertexDataQuad.indices = indicesQuad;

```

```

vertexDataQuad.applyToMesh(quadMesh);

// Matériau avec texture pour prouver que les UV fonctionnent
const matQuad = new BABYLON.StandardMaterial("matQuad", scene);
matQuad.diffuseTexture = new BABYLON.Texture("textures/checkerboard_basecolor.png",
scene);
quadMesh.material = matQuad;

quadMesh.position.x = 1.5;

// =====
// OUTILS PEDAGOGIQUES
// =====
// scene.debugLayer.show(); // Afficher l'inspecteur pour montrer les buffers

return scene;
};
    
```

### 💡 Points interactifs à observer

1. **Le Backface Culling en direct :** Faites pivoter la caméra dans le Playground pour regarder derrière le triangle et le carré. Les objets deviendront invisibles. Cela illustre parfaitement que l'ordre des indices définit une face orientée d'un seul côté, optimisant ainsi le rendu GPU.
2. **Jouer avec le Wireframe :** Décommentez la ligne `matTri.wireframe = true;`. C'est un excellent moyen visuel de prouver que le carré (à droite) est bel et bien composé de deux triangles (la diagonale apparaît).
3. **Casser l'Index Buffer :** Changez `indicesQuad = [0, 1, 2, 0, 2, 3]` par `[0, 1, 2]`. Un seul triangle du carré s'affiche, prouvant le rôle des indices.
4. **Casser les UVs :** Changez la coordonnée UV du point 2 (haut-droite) de 1, 1 à 0.5, 0.5. La texture du damier se déformera instantanément, ce qui permet de constater le fonctionnement du « texture mapping ».

## Session 2 : Transformations et Espaces de Coordonnées

### 🔗 Objectifs de la session

Comprendre le voyage d'un vertex à travers les espaces de coordonnées, maîtriser les matrices de transformation, et exploiter la hiérarchie de scène (parenting) pour déplacer et animer des objets complexes.

## Le voyage d'un Vertex : Le Pipeline de Transformation

### Un vertex possède plusieurs positions

Un vertex ne possède pas « une » position, mais une série de positions selon le référentiel dans lequel on l'observe. À chaque étape, une matrice transforme les coordonnées d'un espace à l'autre.



Fig. 4. – Un vertex voyage d'un espace à l'autre jusqu'à ce qu'il devienne un pixel à l'écran.

### Espace Local (Object Space)

L'origine est au centre de l'objet  $(0, 0, 0)$ . C'est l'espace dans lequel l'artiste modélise le mesh. Les coordonnées sont relatives à l'objet lui-même, indépendamment de sa position dans la scène.

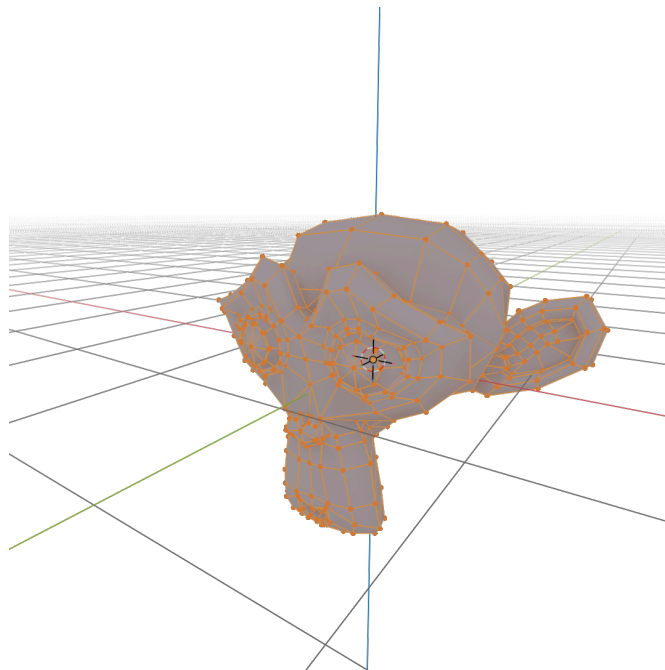


Fig. 5. – Espace Local : l'origine est au centre de l'objet.

### Espace Monde (World Space)

La position de l'objet dans la scène globale. Tous les objets y sont exprimés dans un repère commun.

**Transformation :** Matrice Monde (Model Matrix) — combine la position, rotation et échelle de l'objet pour passer de l'espace local à l'espace monde.

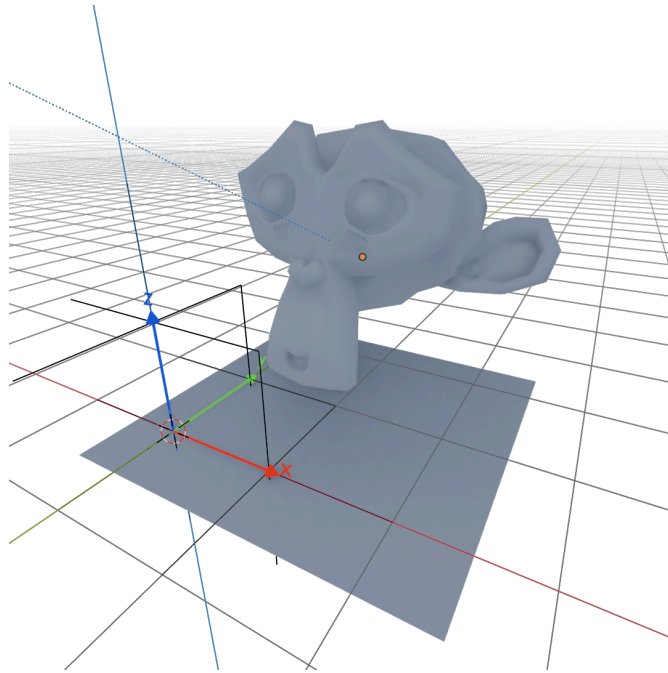


Fig. 6. – Espace Monde : tous les objets partagent un repère commun.

### Espace Caméra (View Space)

Le monde vu depuis l'œil du joueur. La caméra devient l'origine  $(0, 0, 0)$ , et les axes sont alignés sur son orientation.

**Transformation :** Matrice Vue (View Matrix) — transforme tout le monde pour qu'il soit vu depuis la caméra.

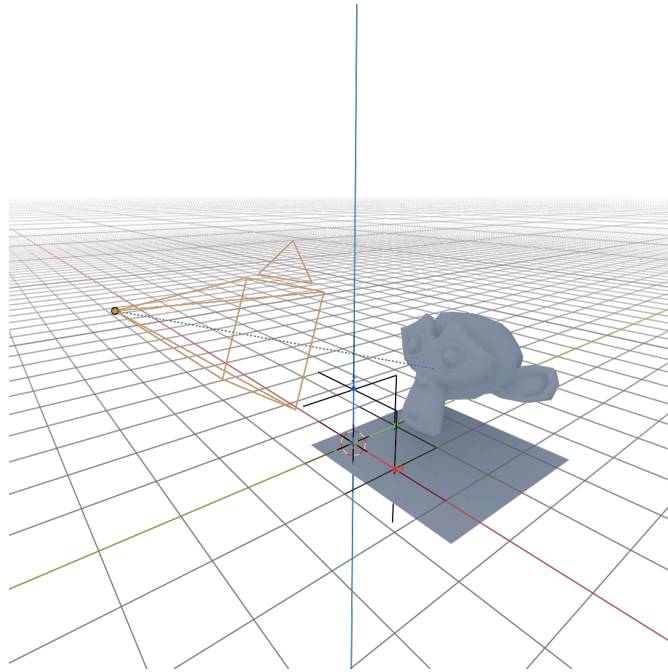


Fig. 7. – Espace Caméra : la caméra devient l'origine, le monde est vu depuis son point de vue.

### Espace Projection (Clip Space)

Conversion du volume 3D en un cube plat  $[-1, 1]$ . C'est ici qu'on définit la perspective : plus un objet est loin, plus il apparaît petit.

**Transformation :** Matrice de Projection — projette les coordonnées 3D en coordonnées 2D normalisées (NDC).

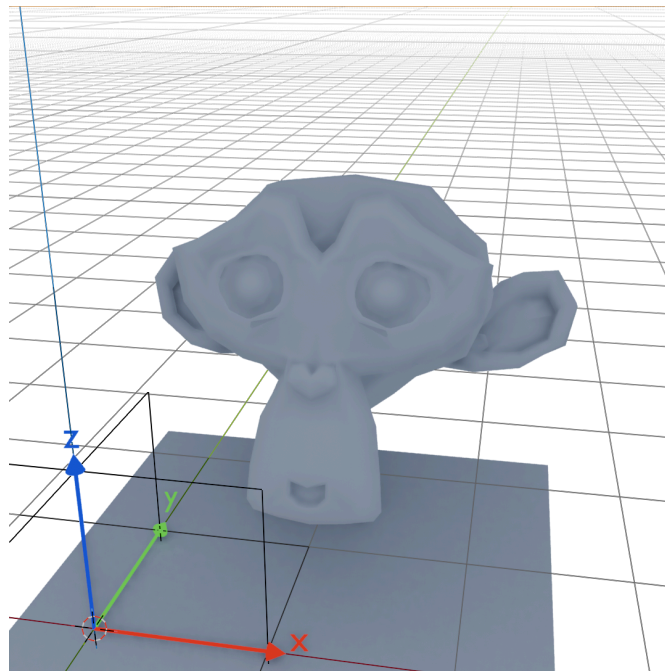


Fig. 8. – Espace Projection : le volume 3D est projeté dans un cube normalisé  $[-1, 1]$ .

## Comprendre les Matrices : L'approche « Calculatrice »

### La matrice comme machine

Pour commencer, oubliez les modèles 3D et les caméras. Ouvrez simplement la console de votre **Playground** Babylon.js. Nous allons observer la matrice pour ce qu'elle est réellement : une simple « machine » mathématique qui prend un point en entrée, le transforme, et recrache un nouveau point.

## La Matrice Identité : La machine au repos

### Définition

La Matrice Identité est une grille 4x4 avec une diagonale de 1 et des 0 partout ailleurs. En algèbre matricielle, c'est l'équivalent du chiffre 1 dans une multiplication classique ( $5 \times 1 = 5$ ).

*Micro-exercice 1 : Prouvez que cette matrice ne fait rien. Consigne :* Créez un point (un Vector3) à la position (3, 4, 5). Créez ensuite une Matrice Identité. Faites passer votre point dans cette matrice à l'aide de la fonction TransformCoordinates et affichez le résultat dans la console.

```
let p = new BABYLON.Vector3(3, 4, 5);
let mIdentite = BABYLON.Matrix.Identity();

// On transforme le point avec la matrice
let resultat = BABYLON.Vector3.TransformCoordinates(p, mIdentite);
console.log("Résultat Identité :", resultat); // Affiche {X:3, Y:4, Z:5}
```

## La Matrice de Scale : Le multiplicateur

### Principe

Si vous remplacez les 1 de la diagonale par d'autres chiffres, la machine devient un multiplicateur. Le premier chiffre touche l'axe X, le deuxième le Y, et le troisième le Z.

*Micro-exercice 2 : Aplatis un point. Consigne :* Prenez le point (2, 2, 2). Fabriquez une matrice de **Scale** qui double sa taille en X et Y, mais l'écrase complètement à 0 sur l'axe Z. Affichez le résultat.

```
let p2 = new BABYLON.Vector3(2, 2, 2);
let mScale = BABYLON.Matrix.Scaling(2, 2, 0); // Z est multiplié par 0 !

let resultatScale = BABYLON.Vector3.TransformCoordinates(p2, mScale);
console.log("Résultat Scale :", resultatScale); // Affiche {X:4, Y:4, Z:0}
```

*Micro-exercice 3 : Le téléporteur. Consigne :* Vous êtes à l'origine (0, 0, 0). Utilisez une matrice de translation pour vous téléporter aux coordonnées (10, 5, -2).

```
let p3 = new BABYLON.Vector3(0, 0, 0);
let mTrans = BABYLON.Matrix.Translation(10, 5, -2);

let resultatTrans = BABYLON.Vector3.TransformCoordinates(p3, mTrans);
console.log("Résultat Translation :", resultatTrans); // Affiche {X:10, Y:5, Z:-2}
```

## Le laboratoire de test : L'ordre des opérations

### Combiner les transformations

Maintenant que vous possédez les briques de base, nous allons combiner ces opérations. C'est ici que le pouvoir de la matrice prend tout son sens : on peut fusionner des dizaines de transformations en une seule « Super Matrice ».

Pour fusionner deux matrices, il faut les multiplier. Mais attention : en mathématiques, l'ordre dans lequel vous multipliez vos matrices change complètement le résultat.  $A \times B \neq B \times A$ .

*Micro-exercice 4 : Le piège de l'ordre.* **Consigne :** Vous avez un point de départ à  $x = 1$ . Vous disposez d'une matrice **Scale**  $\times 2$  et d'une matrice **Translation**  $+3$ .

- **Cas A :** Fusionnez les matrices pour appliquer le **Scale** d'abord, puis la **Translation**. Quel est le résultat ?
- **Cas B :** Fusionnez les matrices pour appliquer la **Translation** d'abord, puis le **Scale**. Quel est le résultat ?

```
let point = new BABYLON.Vector3(1, 0, 0);
let S = BABYLON.Matrix.Scaling(2, 1, 1);
let T = BABYLON.Matrix.Translation(3, 0, 0);

// Cas A : T * S (On scale d'abord, puis on translate)
// En math : (1 * 2) + 3 = 5
let matriceA = S.multiply(T);
let resA = BABYLON.Vector3.TransformCoordinates(point, matriceA);
console.log("Cas A :", resA.x); // Affiche 5

// Cas B : S * T (On translate d'abord, puis on scale)
// En math : (1 + 3) * 2 = 8
let matriceB = T.multiply(S);
let resB = BABYLON.Vector3.TransformCoordinates(point, matriceB);
console.log("Cas B :", resB.x); // Affiche 8
```

### ! Observation clé

Bouger puis grossir ne donne pas la même position finale que grossir puis bouger ! C'est ce qui crée des comportements étranges (les « orbites ») quand vous animez des objets complexes.

## Anatomie des matrices de transformation

### Matrice de Translation

Déplace un point. La 4ème colonne contient les valeurs de déplacement  $(t_x, t_y, t_z)$ .

$$T = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

## Matrice de Scale (Échelle)

Multiplie les coordonnées par un facteur. La diagonale contient  $(s_x, s_y, s_z)$ .

$$S = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

## Matrices de Rotation

Rotation autour de chaque axe. Les axes perpendiculaires sont affectés par  $\cos \theta$  et  $\sin \theta$ .

### Rotation X

$$R_x = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

### Rotation Y

$$R_y = \begin{pmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

### Rotation Z

$$R_z = \begin{pmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

## 🔍 Observation

La matrice de **Translation** utilise la 4ème colonne (d'où l'importance de  $w = 1$ ), tandis que les matrices de **Rotation** et **Scale** utilisent la zone 3x3 supérieure gauche. C'est cette séparation qui rend les coordonnées homogènes indispensables.

## La Matrice de Translation et l'astuce de la coordonnée W

### Pourquoi 4x4 ?

Vous vous demandez sûrement pourquoi nous utilisons des matrices 4x4 pour un espace en 3D (X, Y, Z) ? Le problème est que multiplier la diagonale ne permet pas **d'ajouter** (donc de déplacer) un point. La solution se trouve dans la 4ème colonne de la matrice. Pour que le calcul active cette colonne de déplacement, votre point 3D possède en réalité une 4ème coordonnée cachée :  $w = 1$ . L'opération mathématique finale fera donc : NouveauX =  $(X \times 1) + (\text{TranslationX} \times 1)$ .

### Versions 2D des matrices de transformation

Les mêmes transformations existent en 2D (X, Y) avec des matrices 3x3. On retrouve la même structure : la translation dans la 3ème colonne, le scale sur la diagonale, et la rotation dans la zone 2x2.

#### Translation 2D

$$\begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix}$$

#### Scale 2D

$$\begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

#### Rotation 2D

$$\begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

#### Shear X

$$\begin{pmatrix} 1 & k_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

#### Shear Y

$$\begin{pmatrix} 1 & 0 & 0 \\ k_y & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

## Atelier Desmos : Visualiser les matrices en 2D

### Outil

Ouvrez Desmos, calculatrice matricielle : <https://www.desmos.com/matrix>. Desmos permet de définir des matrices et de les multiplier directement. C'est l'outil idéal pour visualiser l'effet d'une matrice sur une forme géométrique avant de passer au 3D.

**Étape 1 : Définir un point et la matrice Identité. Consigne :** Dans Desmos, créez une matrice  $M = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$  et un point  $P = (2, 3)$ . Multipliez  $M \times P$  pour obtenir un nouveau point  $P'$ .

- Entrez :  $M = \begin{bmatrix} 1, & 0 \\ 0, & 1 \end{bmatrix}$
- Entrez :  $P = (2, 3)$
- Entrez :  $P' = M * P$

**Résultat attendu :**  $P' = (2, 3)$ . La matrice Identité ne change rien — le point reste à la même position.

**Étape 2 : Scale (Étirer la forme). Consigne :** Remplacez la matrice  $M$  par une matrice de scale qui double la taille en X et réduit de moitié en Y.

- Entrez :  $M = \begin{bmatrix} 2, & 0 \\ 0, & 0.5 \end{bmatrix}$
- Observez  $P'$  : le point s'est déplacé de  $(2, 3)$  à  $(4, 1.5)$ .

**Expérience :** Créez un polygone (un carré) avec plusieurs points et appliquez la matrice à chacun. Vous verrez la forme s'étirer et s'aplatir en temps réel quand vous modifiez les valeurs de  $M$ .

**Étape 3 : Rotation (Faire tourner la forme). Consigne :** Utilisez la matrice de rotation 2D avec un angle  $\theta$  que vous pouvez animer.

- Entrez :  $M = \begin{bmatrix} \cos(\theta), & -\sin(\theta) \\ \sin(\theta), & \cos(\theta) \end{bmatrix}$
- Ajoutez un curseur pour  $\theta$  (cliquez sur  $\theta$  puis activez le curseur).
- Créez un carré avec 4 points et appliquez  $M$  à chacun.

**Résultat :** En faisant varier  $\theta$  de 0 à  $2\pi$ , la forme tourne autour de l'origine. C'est exactement ce que fait le GPU avec vos modèles 3D, mais avec une matrice 4x4 et un axe supplémentaire.

**Étape 4 : Combiner Translation + Rotation (Le piège !). Consigne :** En 2D, la translation nécessite une matrice 3x3 avec coordonnées homogènes. Définissez une matrice de translation et une matrice de rotation, puis testez les deux ordres de multiplication.

- **Translation :**  $T = \begin{bmatrix} 1, & 0, & 3 \\ 0, & 1, & 2 \\ 0, & 0, & 1 \end{bmatrix}$
- **Rotation :**  $R = \begin{bmatrix} \cos(\theta), & -\sin(\theta), & 0 \\ \sin(\theta), & \cos(\theta), & 0 \\ 0, & 0, & 1 \end{bmatrix}$
- **Cas A (Rotation puis Translation) :**  $M_A = T * R$
- **Cas B (Translation puis Rotation) :**  $M_B = R * T$

**Observation :** Animez  $\theta$  et comparez les deux trajectoires. Dans le Cas A, la forme tourne sur elle-même puis est déplacée. Dans le Cas B, la forme orbite autour de l'origine. C'est la démonstration visuelle que  $A \times B \neq B \times A$ .

## ! Ce que vous venez de comprendre

Desmos vous a permis de voir, en 2D, exactement ce que le GPU fait en 3D avec des matrices 4x4. Chaque point (vertex) de votre modèle est multiplié par une matrice pour déterminer sa position finale à l'écran. L'ordre des multiplications détermine le comportement de l'animation.

## Le lien avec la géométrie : La boucle est bouclée

### La matrice applique la même règle à tout l'univers

Rappelez-vous : un modèle 3D (Mesh) n'est qu'un immense tableau géant rempli de Vector3. La matrice est la formule magique que l'on va appliquer dans une boucle for sur **absolument tous** ces points.

*Micro-exercice 5 : Incarnez la carte graphique (GPU) !.* **Consigne :** Voici le code du carré plat de la Session 1. Avant d'envoyer ces positions pour créer la face 3D, vous allez faire le travail du Vertex Shader. Créez une matrice de Scale(2, 0.5, 1). Faites passer chaque point de votre tableau (grâce à une boucle) dans cette matrice pour déformer la géométrie à la main.

```
// Les coordonnées de base de notre carré
let positions = [
    new BABYLON.Vector3(-1, -1, 0),
    new BABYLON.Vector3(1, -1, 0),
    new BABYLON.Vector3(1, 1, 0),
    new BABYLON.Vector3(-1, 1, 0)
];

// Notre matrice de transformation (Super Matrice)
let mScale = BABYLON.Matrix.Scaling(2, 0.5, 1);

// Vous faites ici ce que le Vertex Shader fera des millions de fois par seconde :
for(let i = 0; i < positions.length; i++) {
    positions[i] = BABYLON.Vector3.TransformCoordinates(positions[i], mScale);
}

// Vérifions le premier point :
console.log("Ancien bas-gauche (-1,-1,0) est devenu :", positions[0]);
// Affiche {X:-2, Y:-0.5, Z:0}

// À ce stade, vous pouvez extraire ces valeurs (X,Y,Z) et les injecter
// dans le VertexData.positions
```

## ! Félicitations

Vous venez de comprendre la base absolue de la programmation graphique. Quand, plus tard dans le cours, vous utiliserez `monObjet.position.x = 5`, vous saurez que derrière ce code simple, le moteur génère une matrice de translation et l'applique à des milliers de sommets exactement comme vous venez de le faire !

## GPU

Graphics Processing Unit

10496 Cores

Lent - Tâches simples

## CPU

Graphics Processing Unit

24 Cores

Rapide - Tâches complexes

Fig. 9. – Le GPU peut effectuer un grand nombre de multiplications de matrices.

## Hiérarchie de scène : Le Parenting

### Le concept du référentiel local

Les objets 3D sont organisés en hiérarchie parent-enfant. Le concept fondamental à retenir est que l'univers d'un enfant, c'est son parent.

L'origine (0, 0, 0) de l'enfant n'est plus le centre du monde, mais le centre de son parent. Ses axes (X, Y, Z) s'alignent sur ceux de son parent. C'est ce qu'on appelle le **Scene Graph**.

### ❗ Pourquoi ne peut-on pas juste additionner les positions ?

Si un parent est en  $X = 10$  et que l'enfant est en  $X = 2$  par rapport au parent, on pourrait penser que l'enfant est en  $X = 12$  dans le monde.

**C'est faux !** Que se passe-t-il si le parent a tourné de 90 degrés ? L'axe X de l'enfant pointe désormais vers l'axe Z du monde. C'est pour cela que la simple addition ne fonctionne pas.

Le GPU doit obligatoirement utiliser la multiplication de matrices pour calculer la vraie position finale :

$$\text{MatriceMonde}_{\text{Enfant}} = \text{MatriceMonde}_{\text{Parent}} \times \text{MatriceLocale}_{\text{Enfant}}$$

## Atelier : Le Train et le Passager

### Les propriétés de base dans Babylon.js

- `.position` : Toujours **locale** par rapport au parent.
- `.rotation` (ou `.rotationQuaternion`) : Orientation **locale**.
- `.scaling` : Échelle **locale**.

### 💡 L'objectif de l'atelier

Nous allons créer un « Train » (le parent) et un « Passager » (l'enfant). Nous allons observer ce qui se passe dans la console pour comprendre la différence entre la position que vous tapez dans le code (Locale) et la position réelle calculée par le GPU (Monde).

*Micro-exercice 6 : Embarquement immédiat.* **Consigne :** Créez une boîte rectangulaire (`train`) et placez-la à  $X = 10$ . Créez une petite sphère (`passager`). Appliquez la propriété `.parent` pour lier le passager au train. Placez le passager à la position locale  $X = 2$ . Observez les coordonnées.

```
// 1. Le Train (Le repère parent)
const train = BABYLON.MeshBuilder.CreateBox("train", {width: 4, height: 1, depth: 2}, scene);
train.position.x = 10; // Le train est à 10 unités de l'origine du monde

// 2. Le Passager (Le repère enfant)
const passager = BABYLON.MeshBuilder.CreateSphere("passager", {diameter: 0.5}, scene);

// LA MAGIE OPÈRE ICI :
passager.parent = train;

// La position locale du passager (Dans le référentiel du train)
passager.position = new BABYLON.Vector3(2, 1, 0);

// --- OBSERVATION DANS LA CONSOLE ---

// 1. Où est le passager par rapport au train ?
console.log("Position Locale : ", passager.position.x);
// Affiche : 2

// 2. Où est le passager par rapport au monde 3D ?
passager.computeWorldMatrix(true); // On force le moteur à calculer la multiplication des matrices
let worldPos = passager.getAbsolutePosition();

console.log("Position Monde : ", worldPos.x);
// Affiche : 12 (car le train n'a pas de rotation, l'addition fonctionne exceptionnellement ici)
```

**Micro-exercice 7 : Le crash test (Rotation).** **Consigne :** Ajoutez une rotation au train de 90 degrés sur l'axe Y ( $\text{Math.PI} / 2$ ). Affichez à nouveau la position absolue du passager.

```
train.rotation.y = Math.PI / 2; // Tourne le train de 90 degrés

passager.computeWorldMatrix(true);
console.log("Nouvelle Position Monde X : ", passager.getAbsolutePosition().x);
// Affiche : 10 ! (Et non plus 12)

console.log("Nouvelle Position Monde Z : ", passager.getAbsolutePosition().z);
// Affiche : -2 !
```

**Explication :** La position locale du passager est toujours  $x=2$ . Mais comme le train a tourné, l'axe X du train correspond maintenant à l'axe Z négatif du monde. La matrice a calculé cette conversion complexe pour vous !

## Concepts clés à retenir (Les pièges classiques)

### 🔗 L'héritage du Scale (L'effet Œuf)

Testez ceci : Appliquez `train.scaling.x = 2` pour allonger votre train. Que se passe-t-il ?

Votre passager (la sphère) se transforme en œuf aplati ! Pourquoi ? Parce que la matrice monde du train contient un **Scale x2**. En multipliant cette matrice, le GPU multiplie aussi les sommets de l'enfant par 2 sur cet axe.

**Bonne pratique :** En production de jeux vidéo, on évite presque toujours d'appliquer un « Scale » à un objet parent, pour éviter de déformer accidentellement tous ses enfants.

### Transformation de vecteurs vs points

Rappelez-vous la coordonnée  $w$ . Les normales (l'orientation des faces pour la lumière) sont des **vecteurs** ( $w = 0$ ), pas des points ( $w = 1$ ). Elles ne subissent donc **pas** la translation de la matrice — seulement la rotation. C'est pourquoi une face orientée vers le haut pointe toujours vers le haut, même si on déplace l'objet de 1000 kilomètres.

### 💡 Question de réflexion pour la prochaine session

« Si je déplace ma caméra de 5 unités vers la droite, que doit faire le GPU avec tous les objets du monde pour qu'ils apparaissent correctement à l'écran ? »

**Réponse :** Il doit tous les déplacer de 5 unités vers la **gauche**. C'est le rôle de la Matrice Vue (View Matrix) — dans le moteur 3D, la caméra est fixe à l'origine, c'est l'univers entier qui bouge en sens inverse autour d'elle pour simuler le mouvement !

# Session 3 : Création de Shaders dans Babylon.js (Partie 1)

## 🔗 Objectifs de la session

Comprendre ce qu'est un shader, pourquoi le GPU est si rapide, et écrire ses premiers shaders GLSL dans Babylon.js.

## De la copie manuelle à la presse d'imprimerie

### Analogie : Gutenberg et le Shader

Avant Gutenberg, chaque livre était copié à la main, lettre par lettre. C'est comme dessiner un cercle, puis un rectangle, puis une ligne : on donne des instructions une par une, exécutées en série.

Le shader est l'équivalent de la presse d'imprimerie : au lieu de dessiner chaque élément individuellement, on écrit **une fois** un programme (la matrice de caractères), et il s'applique partout simultanément.

## Qu'est-ce qu'un Fragment Shader ?

### Définition

Un shader est un programme exécuté sur la carte graphique (GPU). Contrairement aux API de dessin classiques où l'on dessine cercle, rectangle, triangle un par un, un shader exécute **les mêmes instructions sur chaque pixel de l'écran**.

Le programme est une fonction à laquelle on passe une position et qui renvoie une couleur :

$$\text{couleur} = f(\text{position})$$

Une fois compilé, ce programme s'exécute extrêmement rapidement.

## Pourquoi les shaders sont-ils rapides ?

### CPU : un tuyau, GPU : une grille de tuyaux

Imaginez le CPU comme un gros tuyau : les opérations passent une par une. Les ordinateurs récents ont plusieurs tuyaux (threads), mais ils restent limités.

Le GPU (Graphics Processing Unit) est une **grille** de milliers de petits microprocesseurs qui tournent en parallèle. Chaque pixel est une petite opération ; au lieu de les traiter séquentiellement, le GPU les traite tous simultanément.

### 💬 Le calcul en parallèle

Sur un écran 800x600 à 60 FPS :  $480\,000 \text{ pixels} \times 60 = \mathbf{28\,800\,000 \text{ calculs par seconde}}$ .

Sur un écran Rétina 2880x1800 à 60 FPS :  $\mathbf{311\,040\,000 \text{ calculs par seconde}}$ .

Un CPU seul ne peut pas suivre. Le GPU résout ce problème par le traitement parallèle : des milliers de petits cœurs traitent les pixels simultanément.

### 💡 Bonus : accélération matérielle

Le GPU accélère aussi certaines fonctions mathématiques directement au niveau du matériel : transformations de matrices, fonctions trigonométriques, etc. Ces opérations sont traitées à la vitesse de l'électricité.

## Pourquoi les shaders font peur ?

### Les deux contraintes des threads GPU

Pour fonctionner en parallèle, chaque thread doit être **indépendant** des autres. Cela implique deux contraintes :

- **Cécité** : un thread ne peut pas voir ce que font les autres threads. Il est impossible de vérifier le résultat d'un autre thread ou de transmettre des données entre threads.
- **Amnésie** : dès qu'un thread a fini son traitement, le GPU lui ré-assigne une autre opération. Le thread ne garde **aucune mémoire** de ce qu'il faisait la frame précédente. Il pourrait dessiner un bouton, puis un bout de ciel, puis du texte.

### 💡 Conséquence pour le code

Coder un shader demande un niveau d'abstraction différent : il faut écrire une fonction **générique** qui sait rendre une image entière uniquement à partir de la variation d'une position, sans aucun état persistant.

## Le Pipeline de Rendu Programmable

### Étapes du pipeline

1. **Vertex Shader** : Transforme chaque sommet de l'Object Space au Clip Space.
2. **Rasterization** : Le GPU convertit les triangles en pixels.
3. **Fragment Shader** : Calcule la couleur de chaque pixel (la fonction  $f(\text{position}) \rightarrow \text{couleur}$ ).
4. **Output Merger** : La couleur finale est écrite dans le framebuffer.

## GLSL : Le langage des Shaders

### Qu'est-ce que le GLSL ?

GLSL (OpenGL Shading Language) est le langage pour écrire des shaders OpenGL. Les spécifications sont maintenues par le **Khronos Group**. Il existe d'autres langages selon les plateformes (HLSL pour DirectX, MSL pour Metal), mais nous nous concentrerons sur GLSL.

## Syntaxe GLSL de base

```
// Vertex Shader : transforme la position du sommet
attribute vec3 position;
uniform mat4 worldViewProjection;

void main() {
    gl_Position = worldViewProjection * vec4(position, 1.0);
}

// Fragment Shader : la fonction f(position) -> couleur
// Ici, tous les pixels seront rouges
void main() {
    gl_FragColor = vec4(1.0, 0.0, 0.0, 1.0); // Rouge
}
```

## Décryptage du code

- `gl_FragColor` est une variable **globale** fournie par GLSL. Elle représente la couleur de sortie du pixel. Le GPU lit cette variable à la fin de `main()` et l'écrit dans le framebuffer.
- `vec4(1.0, 0.0, 0.0, 1.0)` est un vecteur à 4 composantes :  $(R, G, B, A)$ . Les valeurs vont de 0.0 à 1.0 (pas de 0 à 255).
  - $R = 1.0$  : rouge au maximum.
  - $G = 0.0, B = 0.0$  : pas de vert ni de bleu.
  - $A = 1.0$  : opacité complète (pixel opaque).
- **Résultat** : tous les pixels de la géométrie sont colorés en rouge pur.

## 💡 Rappelez-vous

Le fragment shader s'exécute sur **chaque pixel** simultanément. Chaque instance est aveugle et amnésique : elle ne connaît que sa propre position. `gl_FragColor` est le seul moyen de communiquer le résultat au GPU.

## Shaders dans Babylon.js : ShaderMaterial

Créer un *ShaderMaterial*.

```
BABYLON.Effect.ShadersStore["customVertexShader"] = `
    attribute vec3 position;
    uniform mat4 worldViewProjection;
    varying vec3 vPosition;

    void main() {
        vPosition = position;
        gl_Position = worldViewProjection * vec4(position, 1.0);
    }
`;
```

```
BABYLON.Effect.ShadersStore["customFragmentShader"] = `
    varying vec3 vPosition;

    void main() {
```

```

        gl_FragColor = vec4(vPosition.x, vPosition.y, vPosition.z, 1.0);
    }
`;

const shaderMaterial = new BABYLON.ShaderMaterial(
    "shader", scene, { vertex: "custom", fragment: "custom" },
    { attributes: ["position"], uniforms: ["worldViewProjection"] }
);
mesh.material = shaderMaterial;

```

## Uniforms et Varyings

### Communication entre shaders

- **Uniforms** : Variables passées du CPU (JavaScript) au GPU (shader). Constantes pour tous les sommets/pixels.
- **Attributes** : Données par sommet (position, normal, UV).
- **Varyings** : Variables passées du Vertex Shader au Fragment Shader (interpolées par pixel).

## Session 4 : Création de Shaders dans Babylon.js (Partie 2)

### 🔗 Objectifs de la session

Découvrir le Node Material Editor de Babylon.js

<https://iquilezles.org/>

### La fonction step

<https://nme.babylonjs.com/#G31DQU>

### La terre vue par NASA

Vous pouvez télécharger des textures de la terre sur ce site : <https://www.solarsystemscope.com/textures/>

<https://www.solarsystemscope.com/textures/#earth>

Vous trouverez le resultat du shader dans ce lien : <https://nme.babylonjs.com/#NSSZS9#1>

<https://nme.babylonjs.com/#NSSZS9#3>

## Session 5 : Lumière et matériaux

### 🔔 Objectifs de la session

Comprendre la mathématique derrière l'éclairage — la loi de Lambert et le produit scalaire  $\vec{N} \cdot \vec{L}$  — puis explorer les sources de lumières de Babylon.js et le modèle spéculaire de Phong/Blinn-Phong.

### 🔔 Rappel : pourquoi la lumière est essentielle

Rappelez-vous la Session 3 : le Fragment Shader calcule la couleur de chaque pixel. Mais sans lumière, un objet 3D n'est qu'un bloc uniforme — tous les pixels ont la même couleur, peu importe la géométrie. C'est la lumière qui révèle la forme, la profondeur et la texture des objets. L'éclairage est le pont entre la géométrie (les normales vues en Session 1) et l'apparence finale (la couleur calculée par le shader).

### 💡 Le rappel critique : les normales

En Session 1, nous avons défini les **normales** — des vecteurs perpendiculaires à chaque face, pointant vers l'extérieur. Ces normales sont l'information essentielle que le GPU utilise pour calculer l'éclairage. Sans normales correctes, la lumière n'a aucun sens. Le produit scalaire ( $\vec{N} \cdot \vec{L}$ ) entre la normale de surface et la direction de la lumière détermine **l'intensité** de la lumière reçue par cette surface.

## Partie 1 : L'Éclairage Mathématique — La Loi de Lambert

### 1. Le problème : combien de lumière reçoit une surface ?

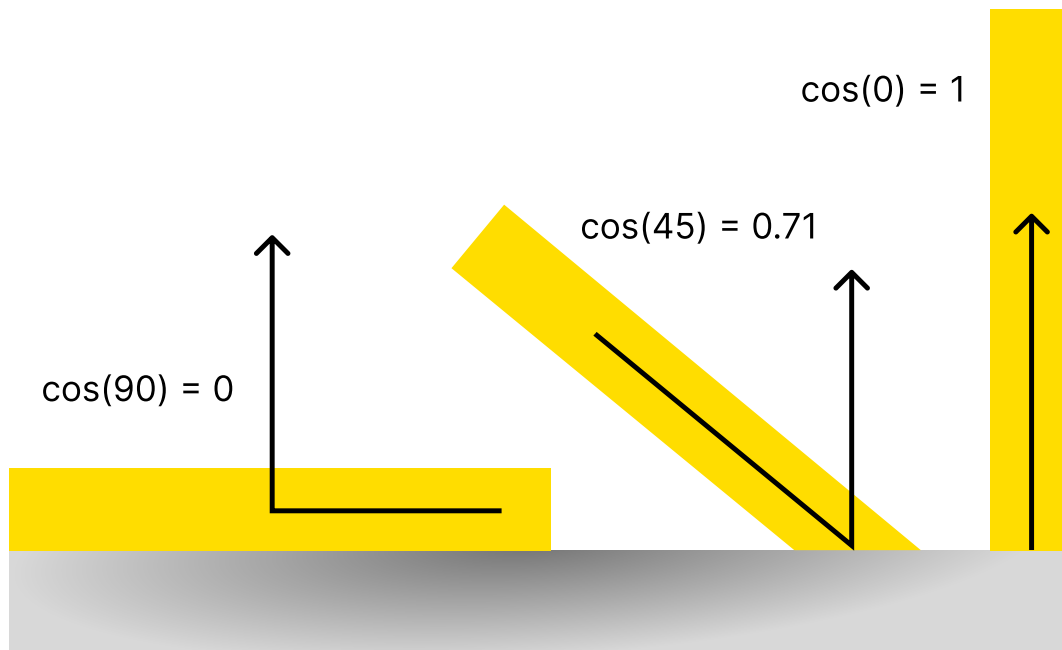


Fig. 10. – Loi de Lambert : L'intensité est proportionnelle au cosinus de l'angle entre la normale  $\vec{N}$  et la direction de la lumière  $\vec{L}$ .

## L'intuition physique

Imaginez une feuille de papier blanche tenue face au soleil. Si vous l'inclinez progressivement, elle devient de plus en plus sombre. Pourquoi ? Parce que la même quantité de lumière se répartit sur une **plus grande surface** projetée — chaque  $\text{cm}^2$  reçoit moins d'énergie. C'est la **loi de Lambert** : l'intensité reçue est proportionnelle au cosinus de l'angle entre la normale de la surface et la direction de la lumière (voir Fig. 10).

## La loi de Lambert (1760)

Pour une surface mate (diffuse), l'intensité lumineuse reçue vaut :

$$k_d = \max(\vec{N} \cdot \vec{L}, 0)$$

où  $\vec{N}$  est la **normale unitaire** de la surface et  $\vec{L}$  la **direction unitaire de la surface vers la lumière**. Le  $\max(\dots, 0)$  coupe les valeurs négatives : une surface dos à la lumière ne reçoit rien.

## 2. Le produit scalaire, au cœur de tout

### Rappel : le produit scalaire

Le produit scalaire de deux vecteurs  $u$  et  $v$  s'écrit :

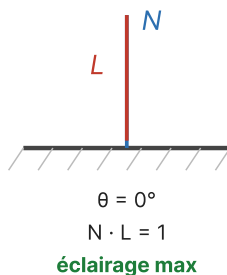
$$u \cdot v = \|u\| \|v\| \cos(\theta)$$

Si  $u$  et  $v$  sont **normalisés** ( $\|u\| = \|v\| = 1$ ), alors :

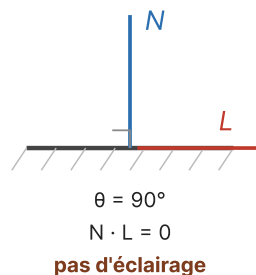
$$u \cdot v = \cos(\theta)$$

C'est la formule fondamentale de toute l'informatique graphique temps réel : un simple produit scalaire donne le cosinus de l'angle entre deux vecteurs.

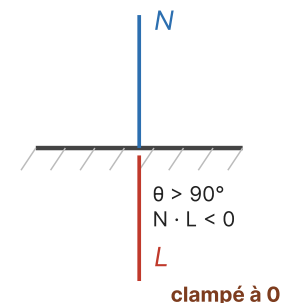
(a) Surface face à la lumière



(b) Lumière rasante



(c) Surface dos à la lumière



Les trois cas de la loi de Lambert :  $N$  (bleu) est la normale,  $L$  (rouge) est la direction de la lumière.

Fig. 11. – Les trois cas de la loi de Lambert : (a) surface face à la lumière, (b) lumière rasante, (c) surface dos à la lumière.

### Les trois cas de la loi de Lambert

- Si  $\vec{N}$  et  $\vec{L}$  pointent dans la même direction (surface face à la lumière) :  $\theta = 0$ ,  $\vec{N} \cdot \vec{L} = 1$ , éclairage **maximal**.
- Si  $\vec{N}$  est perpendiculaire à  $\vec{L}$  (lumière rasante) :  $\theta = 90^\circ$ ,  $\vec{N} \cdot \vec{L} = 0$ , **pas d'éclairage**.
- Si  $\vec{N}$  pointe à l'opposé de  $\vec{L}$  (surface dos à la lumière) :  $\theta > 90^\circ$ ,  $\vec{N} \cdot \vec{L} < 0$ , mais on utilise  $\max(\dots, 0) \rightarrow$  **zéro** (voir Fig. 11).

C'est pourquoi une sphère éclairée présente un dégradé continu du côté éclairé vers le côté sombre : chaque fragment a une normale différente, donc un  $\vec{N} \cdot \vec{L}$  différent.

## 3. La couleur finale : multiplication lumière $\times$ matériau

### La formule complète du diffus

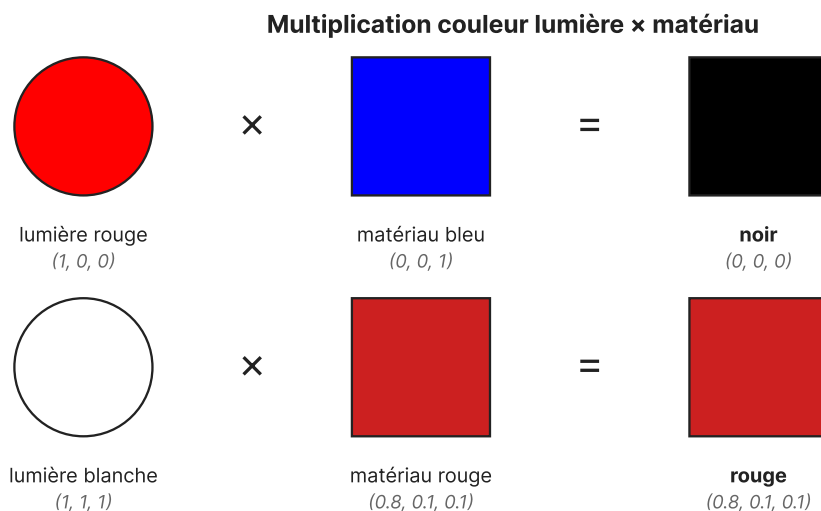
La couleur finale d'un fragment éclairé par la lumière diffuse est :

$$\text{color}_{\text{frag}} = \text{color}_{\text{light}} \times \text{color}_{\text{material}} \times \max(\vec{N} \cdot \vec{L}, 0)$$

- $\text{color}_{\text{light}}$  : couleur de la source (ex. blanc (1, 1, 1)).
- $\text{color}_{\text{material}}$  : couleur intrinsèque de la surface (ex. rouge (1, 0, 0)).
- $\max(\vec{N} \cdot \vec{L}, 0)$  : le facteur d'angle de Lambert.

### 💡 Astuce : la couleur de la lumière se multiplie

La couleur de la lumière (diffuse) se **multiplie** avec la couleur du matériau. Une lumière rouge (1, 0, 0) sur un matériau bleu (0, 0, 1) donne... du noir (0, 0, 0) ! Le bleu ne réfléchit pas le rouge. C'est la physique réelle : un objet bleu sous lumière rouge apparaît noir (voir Fig. 12).



*Un objet bleu sous lumière rouge apparaît noir : le bleu ne réfléchit pas le rouge.*

Fig. 12. – La couleur finale est le produit composante par composante de la couleur de la lumière et de la couleur du matériau.

## 4. Écriture en GLSL : le shader de Lambert

*Fragment shader minimal (Lambert seul).*

```
// Varyings interpolés depuis le vertex shader
in vec3 vNormal;          // Normale du fragment (espace monde)
in vec3 vLightDir;        // Direction de la lumière vers le fragment

uniform vec3 uLightColor;  // Couleur de la lumière
uniform vec3 uMaterialColor; // Couleur du matériau

out vec4 fragColor;

void main() {
    // 1. Normaliser (les varyings ne sont plus unitaires après interpolation)
    vec3 N = normalize(vNormal);
    vec3 L = normalize(vLightDir);

    // 2. Produit scalaire = cos(theta), clampé à [0, 1]
    float diffuse = max(dot(N, L), 0.0);

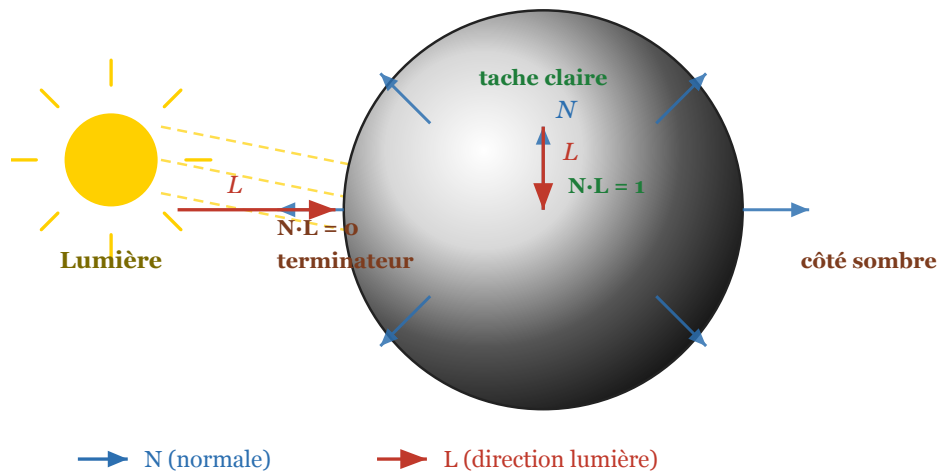
    // 3. Couleur finale = lumière × matériau × facteur d'angle
    vec3 color = uLightColor * uMaterialColor * diffuse;

    fragColor = vec4(color, 1.0);
}
```

### ❗ Pourquoi normaliser dans le fragment shader ?

Les normales sont interpolées **linéairement** entre les sommets du triangle. Cette interpolation ne préserve pas la norme : un vecteur unitaire aux sommets ne l'est plus forcément au milieu du fragment. Il faut donc **toujours** appeler `normalize(N)` dans le fragment shader, même si les normales étaient unitaires dans le vertex shader. Oublier cette étape est un bug classique qui donne un éclairage trop terne ou trop brillant.

## 5. Pourquoi Lambert fait un dégradé sur une sphère



*Le dégradé sur la sphère vient de la variation de  $N \cdot L$  : chaque fragment a une normale différente.*

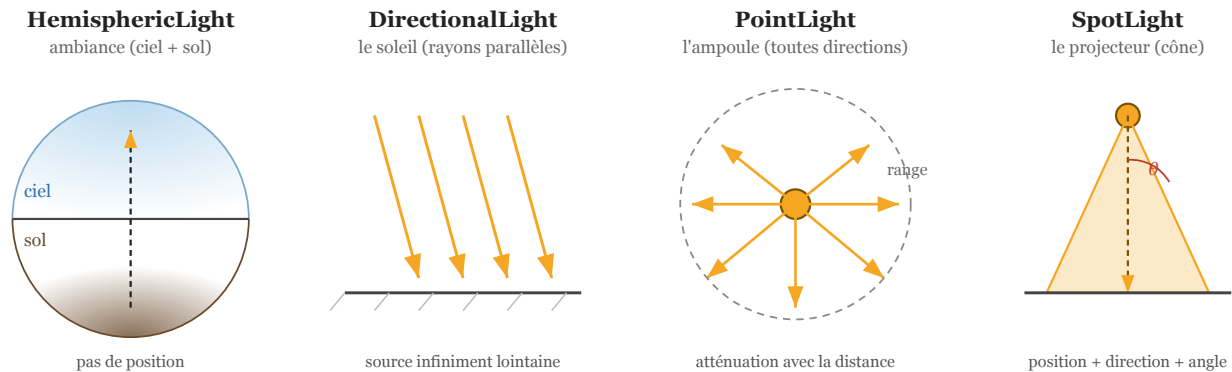
Fig. 13. – Une sphère éclairée : chaque fragment a une normale radiale différente. Au centre éclairé  $\vec{N} \cdot \vec{L} = 1$  (tache claire), au terminator  $\vec{N} \cdot \vec{L} = 0$ , et le côté opposé reste sombre.

### Géométrie d'une sphère éclairée

Sur une sphère, chaque fragment a une normale différente : elle pointe radialement vers l'extérieur. Au centre de la face éclairée,  $\vec{N}$  et  $\vec{L}$  sont alignés ( $\vec{N} \cdot \vec{L} = 1$ ) → tache la plus claire. Au bord,  $\vec{N}$  est perpendiculaire à  $\vec{L}$  ( $\vec{N} \cdot \vec{L} = 0$ ) → ombre. Entre les deux,  $\vec{N} \cdot \vec{L}$  varie doucement de 1 à 0, créant le dégradé caractéristique (voir Fig. 13). C'est exactement ce que reproduit le shader ci-dessus, fragment par fragment.

## Partie 2 : Sources de Lumières et Spéculaire

### 6. Les quatre types de lumières dans Babylon.js



Les quatre types de lumières intégrées de Babylon.js.

Fig. 14. – Les quatre types de lumières intégrées de Babylon.js : HemisphericLight (ambiance), DirectionalLight (soleil), PointLight (ampoule), SpotLight (projecteur).

#### HemisphericLight — La lumière ambiante

Simule la lumière du ciel et du sol. Elle n'a pas de position — elle éclaire toutes les surfaces en fonction de leur orientation (la normale).

- **Ciel** : Couleur pour les surfaces pointant vers le haut (groundColor inversé).
- **Sol** : Couleur pour les surfaces pointant vers le bas.
- **Direction** : Définit quel axe sépare le « haut » du « bas ».
- **Usage** : Lumière d'ambiance de base, évite que les zones non éclairées soient totalement noires.

Créer une *HemisphericLight*.

```
const hemiLight = new BABYLON.HemisphericLight("hemi", new BABYLON.Vector3(0, 1, 0), scene);

// Couleur du ciel (surfaces vers le haut)
hemiLight.diffuse = new BABYLON.Color3(0.8, 0.9, 1.0); // Bleu pâle

// Couleur du sol (surfaces vers le bas)
hemiLightgroundColor = new BABYLON.Color3(0.3, 0.2, 0.1); // Brun

// Couleur spéculaire
hemiLight.specular = new BABYLON.Color3(0, 0, 0); // Pas de reflet spéculaire

hemiLight.intensity = 0.6;
```

#### DirectionalLight — Le soleil

Lumière parallèle qui simule une source infiniment lointaine (comme le soleil). Tous les rayons arrivent dans la même direction. Elle n'a pas de position — seule la direction compte.

- **Direction** : Un vecteur normalisé qui indique la direction des rayons.
- **Pas de portée** : Éclaire toute la scène uniformément.
- **Usage** : Soleil, lumière naturelle, ombres nettes.

*Créer une DirectionalLight.*

```
const sunLight = new BABYLON.DirectionalLight("sun", new BABYLON.Vector3(-1, -2, -1), scene);

sunLight.diffuse = new BABYLON.Color3(1, 0.95, 0.8); // Lumière chaude
sunLight.specular = new BABYLON.Color3(1, 1, 1);      // Reflet blanc
sunLight.intensity = 1.2;

// La direction est normalisée automatiquement par Babylon.js
// mais il est recommandé de la normaliser soi-même :
sunLight.direction.normalize();
```

### PointLight — L'ampoule

Lumière ponctuelle qui émet dans toutes les directions à partir d'une position dans l'espace. Plus un objet est loin, moins il est éclairé (atténuation).

- **Position** : Emplacement de la source dans la scène.
- **Range** : Portée maximale de la lumière. Au-delà, l'intensité tombe à zéro.
- **Atténuation** : L'intensité diminue avec la distance selon une courbe.
- **Usage** : Ampoule, torche, feu, lampe de poche (sans cône).

*Créer une PointLight.*

```
const bulb = new BABYLON.PointLight("bulb", new BABYLON.Vector3(0, 3, 0), scene);

bulb.diffuse = new BABYLON.Color3(1, 0.9, 0.7); // Lumière chaude
bulb.specular = new BABYLON.Color3(1, 1, 1);
bulb.intensity = 1.0;
bulb.range = 15; // Portée de 15 unités

// On peut animer la position pour simuler un mouvement
scene.registerBeforeRender(() => {
    bulb.position.x = Math.sin(performance.now() * 0.001) * 3;
});
```

### SpotLight — Le projecteur

Lumière conique qui combine une position et une direction, avec un angle d'ouverture. C'est la lumière la plus complexe des quatre.

- **Position** : Origine du cône.
- **Direction** : Axe central du cône.
- **Angle** : Angle d'ouverture du cône (en radians, demi-angle).
- **Exponent (penumbra)** : Contrôle le flou du bord du cône. Plus la valeur est élevée, plus le bord est net.
- **Range** : Portée maximale.

- **Usage** : Lampe torche, projecteur de scène, phare de voiture.

Créer une *SpotLight*.

```
const spotlight = new BABYLON.SpotLight(
    "spot",
    new BABYLON.Vector3(0, 5, 0),    // Position (en haut)
    new BABYLON.Vector3(0, -1, 0),   // Direction (vers le bas)
    Math.PI / 4,                    // Angle : 45 degrés
    2,                              // Exponent (penumbra)
    scene
);

spotlight.diffuse = new BABYLON.Color3(1, 1, 0.8);
spotlight.intensity = 2.0;
spotlight.range = 20;
```

## 7. Paramètres communs à toutes les lumières

### Propriétés partagées

- `light.diffuse` : Couleur principale de la lumière (`Color3`). C'est la couleur qui sera mélangée avec la couleur diffuse du matériau.
- `light.specular` : Couleur du reflet spéculaire (`Color3`). Souvent blanc, mais peut être coloré pour des effets créatifs.
- `light.intensity` : Multiplicateur d'intensité (float). Valeur par défaut : 1.0.
- `light.range` : Portée de la lumière (float). Utilisé par `PointLight` et `SpotLight`. Ignoré par `HemisphericLight` et `DirectionalLight`.
- `light.direction` : Direction de la lumière (`Vector3`). Utilisé par `HemisphericLight`, `DirectionalLight`, et `SpotLight`.

## 8. La lumière spéculaire (Modèle de Phong)

### Au-delà du diffus : la brillance

La loi de Lambert explique les surfaces **mates** (bois, papier, béton). Mais une bille de verre, une voiture peinte ou un métal poli réfléchissent aussi la lumière comme un miroir — c'est la composante **spéculaire**. Elle ne dépend pas seulement de l'angle  $\vec{N} \cdot \vec{L}$ , mais aussi de la position de la **caméra** : le reflet se déplace quand on tourne autour de l'objet.

### Le reflet : $\vec{R} \cdot \vec{V}$

La composante spéculaire simule le reflet brillant d'une lumière sur une surface lisse. Elle dépend de trois vecteurs :

- $\vec{L}$  : Direction de la surface vers la lumière.
- $\vec{R}$  : Rayon réfléchi (le miroir de  $\vec{L}$  par rapport à la normale  $\vec{N}$ ).
- $\vec{V}$  : Direction de la surface vers la caméra.

$$k_s = \max(\vec{R} \cdot \vec{V}, 0)^{\text{shininess}}$$

L'exposant "shininess" contrôle la taille du reflet :

- Valeur faible (ex: 2) : reflet large et flou (plastique mat).
- Valeur élevée (ex: 128) : reflet petit et net (métal poli).

### Le vecteur réfléchi $\vec{R}$

Le rayon réfléchi se calcule par la formule :

$$\vec{R} = 2(\vec{N} \cdot \vec{L})\vec{N} - \vec{L}$$

Géométriquement : on décompose  $\vec{L}$  en une composante normale  $(\vec{N} \cdot \vec{L})\vec{N}$  et une composante tangente. La réflexion inverse la composante tangente, donc on double la composante normale et on soustrait  $\vec{L}$ . C'est la loi de la réflexion de Descartes — l'angle d'incidence égal l'angle de réflexion (voir Fig. 15).

*Note* : cette formule n'a de sens que lorsque  $\vec{N} \cdot \vec{L} > 0$  (surface face à la lumière). Quand  $\vec{N} \cdot \vec{L} < 0$ , la surface est dos à la lumière et ne reçoit rien — le shader masque alors la spéculaire avec  $\text{step}(0.0, \text{NdotL})$ .

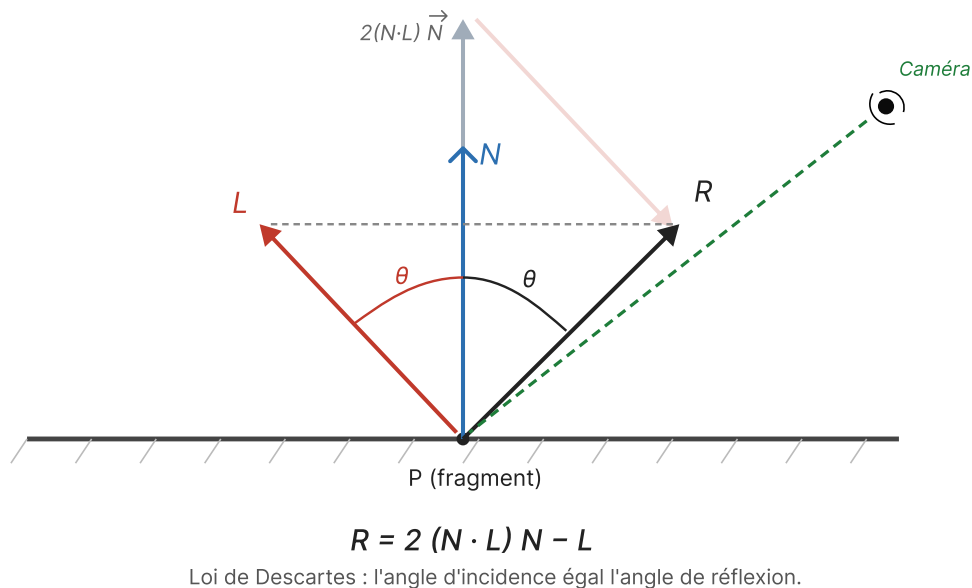


Fig. 15. – Le vecteur réfléchi  $\vec{R} = 2(\vec{N} \cdot \vec{L})\vec{N} - \vec{L}$  : symétrique de  $\vec{L}$  par rapport à la normale  $\vec{N}$ . L'angle d'incidence  $\theta$  égal l'angle de réflexion.

## 9. Blinn-Phong : la variante optimisée

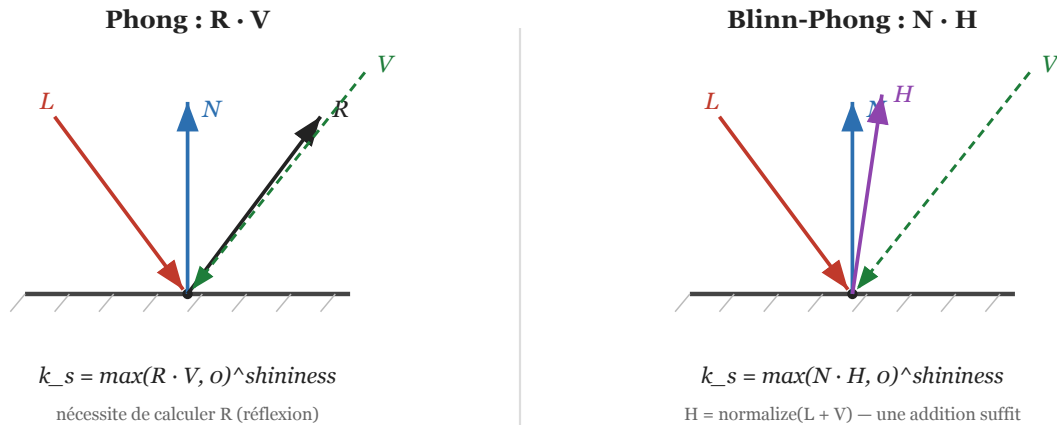
### Le vecteur half-angle $\vec{H}$

Le modèle Blinn-Phong remplace le vecteur réfléchi  $\vec{R}$  par le vecteur **half-angle**  $\vec{H}$  :

$$\vec{H} = \text{normalize}(\vec{L} + \vec{V})$$

$$k_s = \max(\vec{N} \cdot \vec{H}, 0)^{\text{shininess}}$$

**Avantage :**  $\vec{H}$  est plus simple à calculer que  $\vec{R}$  (pas besoin de réfléchir le vecteur — une addition + une normalisation suffit). C'est le modèle utilisé par défaut dans la plupart des moteurs temps réel, y compris le StandardMaterial de Babylon.js (voir Fig. 16).



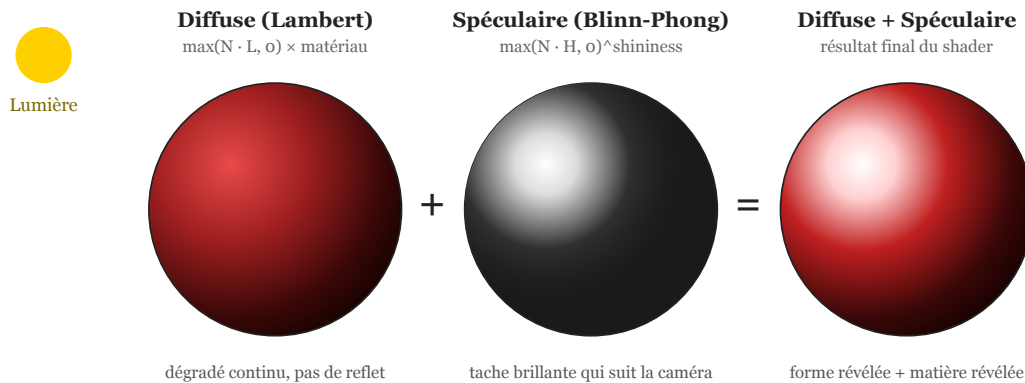
*Blinn-Phong remplace  $R$  (coûteux) par  $H = \text{normalize}(L + V)$ . Visuellement équivalent, plus rapide.*

Fig. 16. – Comparaison Phong vs Blinn-Phong. À gauche, on calcule le vecteur réfléchi  $\vec{R}$  et on le compare à  $\vec{V}$ . À droite, on calcule le vecteur half-angle  $\vec{H} = \text{normalize}(\vec{L} + \vec{V})$  et on le compare à  $\vec{N}$ .

### 💡 Pourquoi Blinn-Phong a remplacé Phong

Le produit scalaire  $\vec{N} \cdot \vec{H}$  donne un angle **deux fois plus petit** que  $\vec{R} \cdot \vec{V}$ . Pour retrouver le même reflet visuel, on multiplie généralement shininess par 4 quand on passe de Phong à Blinn-Phong. Le résultat est visuellement très proche, mais le calcul est plus rapide — un gain précieux quand on rend des millions de fragments par frame.

## 10. Le shader complet : diffus + spéculaire



*La composante diffuse révèle la couleur et la forme ; la spéculaire révèle la brillance et la matière.*

Fig. 17. – La composante diffuse (Lambert) révèle la couleur et la forme ; la composante spéculaire (Blinn-Phong) révèle la brillance et la matière. La somme donne le rendu final.

*Fragment shader Phong (Lambert + spéculaire).*

```
in vec3 vNormal;
in vec3 vLightDir;
in vec3 vViewDir;      // Direction de la caméra vers le fragment

uniform vec3 uLightColor;
uniform vec3 uMaterialColor;
uniform vec3 uSpecularColor; // Couleur du reflet (souvent blanc)
uniform float uShininess;    // Exposant de brillance

out vec4 fragColor;

void main() {
    vec3 N = normalize(vNormal);
    vec3 L = normalize(vLightDir);
    vec3 V = normalize(vViewDir);

    // Produit scalaire brut (non clampé) – sert pour diffuse ET pour le masque
    float NdotL = dot(N, L);

    // --- Composante diffuse (Lambert) ---
    float diffuse = max(NdotL, 0.0);

    // --- Composante spéculaire (Phong) ---
    vec3 R = reflect(-L, N); // R = 2*(N.L)*N - L
    // step(0.0, NdotL) vaut 1 si NdotL >= 0, sinon 0 → masque sans branchement
    float specular = pow(max(dot(R, V), 0.0), uShininess)
        * step(0.0, NdotL);

    // --- Combinaison ---
    vec3 color = uLightColor * uMaterialColor * diffuse
        + uLightColor * uSpecularColor * specular;
```

```
fragColor = vec4(color, 1.0);
}
```

### ❗ Pourquoi `step()` plutôt qu'un `if` ?

Sans masque, la spéculaire apparaîtrait sur la face **non éclairée** — un reflet fantôme. Mais un `if (diffuse <= 0.0)` est un **branchement sur une valeur varying** : comme `diffuse` diffère d'un fragment à l'autre, les threads d'un même warp divergent et s'exécutent en série, ce qui annule l'avantage du SIMD.

La fonction `step(edge, x)` retourne 0 si  $x < \text{edge}$ , 1 sinon — c'est une opération **mathématique pure**, exécutée en lockstep par tous les threads. Multiplier la spéculaire par `step(0.0, NdotL)` coupe donc le reflet sur les faces sombres **sans aucune divergence**. C'est l'idiome standard en GLSL pour tout masque conditionnel.

*Variante Blinn-Phong (plus rapide).*

```
// ... même setup que ci-dessus ...

void main() {
    vec3 N = normalize(vNormal);
    vec3 L = normalize(vLightDir);
    vec3 V = normalize(vViewDir);

    float NdotL = dot(N, L);

    // --- Diffuse (Lambert) ---
    float diffuse = max(NdotL, 0.0);

    // --- Spéculaire (Blinn-Phong) ---
    vec3 H = normalize(L + V); // Vecteur half-angle
    float specular = pow(max(dot(N, H), 0.0), uShininess)
        * step(0.0, NdotL); // Masque sans branchement

    vec3 color = uLightColor * uMaterialColor * diffuse
        + uLightColor * uSpecularColor * specular;

    fragColor = vec4(color, 1.0);
}
```

## 11. StandardMaterial : assembler les deux en Babylon.js

### Deux approches d'éclairage

Babylon.js propose deux principaux types de matériaux pour interagir avec la lumière :

- **StandardMaterial** : Modèle empirique (Lambert + Blinn-Phong). Simple, rapide, mais peu réaliste. Propriétés : `diffuseColor`, `specularColor`, `specularPower`.
- **PBRMaterial** : Modèle physiquement correct (Physically Based Rendering). Plus coûteux, mais produit des résultats réalistes. Propriétés : `albedoColor`, `metallic`, `roughness`. (Vu en détail dans une session ultérieure.)

*StandardMaterial : Lambert + Blinn-Phong en une ligne.*

```
const mat = new BABYLON.StandardMaterial("mat", scene);

// Composante diffuse (Loi de Lambert)
mat.diffuseColor = new BABYLON.Color3(0.8, 0.2, 0.2); // Rouge

// Composante spéculaire (Blinn-Phong)
mat.specularColor = new BABYLON.Color3(1, 1, 1); // Reflet blanc
mat.specularPower = 32; // Shininess

// Quand vous réglez specularPower, vous contrôlez directement
// l'exposant uShininess du shader vu ci-dessus.
```

## Concepts clés à retenir

### 💡 Les 5 points essentiels

1. **La normale est reine** : Sans normales correctes, pas d'éclairage correct. Le produit scalaire  $\vec{N} \cdot \vec{L}$  est la formule fondamentale.
2. **Lambert = cosinus de l'angle** : L'intensité diffuse est  $\max(\vec{N} \cdot \vec{L}, 0)$ . Un simple produit scalaire de vecteurs normalisés donne le cosinus — c'est toute la magie.
3. **La couleur se multiplie** :  $\text{color}_{\text{final}} = \text{color}_{\text{light}} \times \text{color}_{\text{material}} \times \max(\vec{N} \cdot \vec{L}, 0)$ . Une lumière rouge sur un objet bleu donne du noir.
4. **Quatre lumières, quatre usages** : HemisphericLight (ambiance), DirectionalLight (soleil), PointLight (ampoule), SpotLight (projecteur).
5. **Spéculaire = reflet qui dépend de la caméra** : Contrairement au diffus, le spéculaire bouge quand on tourne autour de l'objet. Blinn-Phong ( $\vec{N} \cdot \vec{H}$ ) est l'optimisation standard de Phong ( $\vec{R} \cdot \vec{V}$ ).

### 💡 Question de réflexion pour la prochaine session

« Nous savons maintenant éclairer des objets. Mais que se passe-t-il quand un objet bloque la lumière d'un autre ? Comment le GPU sait-il qu'un fragment est **derrière** un autre vu depuis la lumière ? »

**Réponse** : C'est le **Shadow Mapping**, que nous verrons en Session 6. Le GPU rend la scène depuis le point de vue de la lumière pour calculer ce qui est visible et ce qui est dans l'ombre.

## Session 6 : L'Éclairage Statique

### 🔗 Objectifs de la session

Comprendre pourquoi l'éclairage temps réel est trop coûteux pour les grandes scènes, et comment les moteurs modernes précalculent l'éclairage hors ligne (baking). Découvrir les lightmaps, l'Ambient Occlusion, les light probes, et les stratégies d'éclairage mixte. Les exemples pratiques s'appuient sur la documentation de Godot, mais les concepts sont universels — ils s'appliquent à Unity, Unreal Engine, ou tout moteur custom.

## Partie 1 : Le Problème — L'Éclairage Temps Réel est Trop Cher

### Rappel : le coût de l'éclairage dynamique

En Session 5, nous avons vu que chaque fragment calcule  $\vec{N} \cdot \vec{L}$  pour chaque lumière. Pour une scène avec 1000 fragments et 10 lumières, c'est 10 000 produits scalaires — et une scène réelle a des **millions** de fragments. Ajouter les ombres (shadow maps), la spéculaire, et l'atténuation, et le budget de 2 ms explose rapidement.

### 💡 Le constat des moteurs professionnels

Dans un jeu réel, la majorité de l'éclairage est **statique** : les bâtiments, les murs, le sol ne bougent pas. La lumière du soleil ne change pas (ou rarement). Pourquoi recalculer le même éclairage 60 fois par seconde ? La réponse : on **ne le fait pas**. On le calcule **une fois** hors ligne, on stocke le résultat dans des textures, et on l'affiche gratuitement à l'exécution.

## Partie 2 : Les Lightmaps — L'Éclairage Cuit au Four

### Principe du Lightmapping

Un **lightmap** est une texture qui stocke l'éclairage précalculé pour chaque surface de la scène. Au lieu de calculer  $\vec{N} \cdot \vec{L}$  en temps réel, le GPU se contente de **lire** la couleur dans la texture — une opération  $O(1)$ , quasiment gratuite.

Le processus se déroule en deux phases :

1. **Offline (baking)** : Un algorithme de ray tracing ou de radiance calcule l'éclairage de chaque surface et l'écrit dans une texture UV.
2. **Runtime (rendering)** : Le fragment shader multiplie la couleur du matériau par la couleur du lightmap. Aucun calcul de lumière.

### 1. UV2 — Le dépliage de lightmap

#### Pourquoi un deuxième canal UV ?

Les UV1 servent à appliquer la texture du matériau (diffuse, normal map). Mais une lightmap a besoin d'un **dépliage différent** : chaque surface doit avoir sa propre région dans la texture, sans chevauchement, et avec une densité d'échantillonnage uniforme. C'est le rôle des **UV2** — un second canal UV dédié au lightmap.

- **UV1** : Optimisé pour la répétition de textures (tiling), peut chevaucher.
- **UV2** : Optimisé pour l'éclairage, **aucun chevauchement**, densité uniforme.

### 💡 Dans Godot

Godot génère automatiquement les UV2 lors du baking si l'option « Generate UV2 » est activée sur le MeshInstance3D. On peut aussi fournir des UV2 personnalisés depuis Blender par exemple. Voir la documentation Godot : MeshInstance3D GI Mode.

## 2. Le shader de lightmap — si simple

*Fragment shader avec lightmap (GLSL générique).*

```
in vec2 vUV;           // UV1 pour la texture du matériau
in vec2 vUV2;          // UV2 pour le lightmap
uniform sampler2D uDiffuseMap;
uniform sampler2D uLightmap;

out vec4 fragColor;

void main() {
    vec3 albedo = texture(uDiffuseMap, vUV).rgb;
    vec3 bakedLight = texture(uLightmap, vUV2).rgb;
    // Une simple multiplication — c'est tout !
    fragColor = vec4(albedo * bakedLight, 1.0);
}
```

### 💬 Pourquoi c'est si rapide

Le shader ci-dessus fait **deux lectures de texture et une multiplication**. Aucun produit scalaire, aucune boucle sur les lumières, aucune shadow map. C'est le shader d'éclairage le plus rapide qui existe — et il produit un rendu de qualité **ray tracing** (ombres douces, rebonds de lumière, color bleeding). Le coût : tout est précalculé et immobile.

## 3. Limitations des lightmaps

### ⚠️ Ce que les lightmaps ne peuvent pas faire

- **Objets en mouvement** : Un personnage qui marche sur un sol éclairé par lightmap ne reçoit pas l'éclairage du lightmap — il faut un éclairage dynamique séparé.
- **Lumières dynamiques** : Si une lumière s'allume ou s'éteint, le lightmap ne change pas. Il faut rebaker.
- **Mémoire** : Les lightmaps sont des textures, souvent en HDR, qui consomment de la VRAM. Une grande scène peut nécessiter plusieurs lightmaps de 2048×2048.
- **Temps de baking** : Le précalcul peut prendre de quelques secondes à plusieurs heures selon la complexité de la scène et la qualité demandée.

## 4. Lightmaps dans Godot

### Le système LightmapGI

Godot utilise le noeud `LightmapGI` pour précalculer l'éclairage. Le baking utilise une approche par voxel + ray tracing :

- **Bake Quality** : Low, Medium, High, Ultra — contrôle la densité des voxels et le nombre de rayons.
- **Directional** : Si activé, stocke aussi la direction de l'éclairage (normales de lightmap), ce qui permet aux normal maps d'affecter l'éclairage baked.
- **Interior** : Optimise pour les scènes d'intérieur (pas de lumière du ciel).

Le résultat est stocké dans un fichier `.exr` (HDR) et appliqué automatiquement aux meshes dont le `GI Mode` est réglé sur `Static`.

### 💡 Documentation Godot

- `LightmapGI` : `LightmapGI Class`
- Guide d'utilisation : `Global Illumination in Godot`

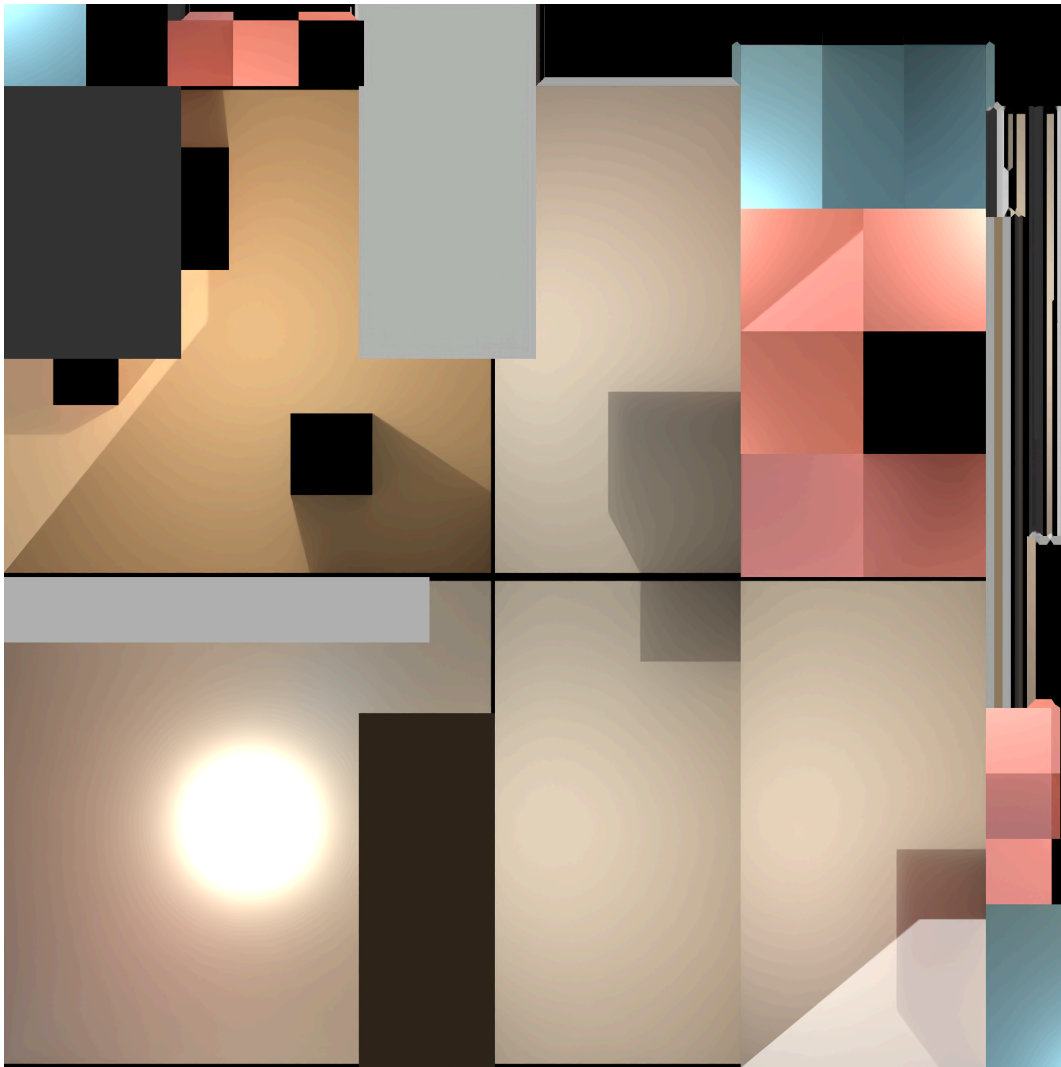


Fig. 18. – Exemple de lightmap baked dans Blender (Cycles, Combined bake, 2048×2048). Chaque îlot UV correspond à une surface de la scène. On y voit l'éclairage direct (taches claires), les ombres portées, et l'occlusion ambiante dans les coins. Cette texture est multipliée par l'albedo au runtime — aucune lumière temps réel nécessaire.

## Partie 3 : L'Ambient Occlusion (AO)

### Qu'est-ce que l'Ambient Occlusion ?

L'Ambient Occlusion simule l'assombrissement qui se produit dans les crevasses, les coins et les zones où la géométrie bloque la lumière ambiante. C'est un phénomène **global** : il ne dépend pas d'une lumière particulière, mais de la géométrie environnante.

$$AO(p) = \text{ratio de directions non bloquées vers l'hémisphère au-dessus de } p$$

Plus une surface est entourée d'autres surfaces, plus l'AO est faible (sombre). Les coins de murs, les plis de tissu, les articulations mécaniques sont des zones de fort AO.

## 5. AO Cuit (Baked AO) vs SSAO

### Deux approches pour l'AO

- **Baked AO** : Précalculé hors ligne et stocké dans une texture (comme un lightmap). Qualité parfaite, coût runtime nul. Mais statique — ne s'adapte pas aux objets en mouvement.
- **SSAO (Screen Space Ambient Occlusion)** : Calculé en temps réel dans l'espace écran en utilisant le depth buffer. Approximation, mais dynamique — réagit aux objets en mouvement. Coût GPU non négligeable.

### 💡 Dans Godot

- **Baked AO** : Intégré automatiquement dans le LightmapGI — l'occlusion ambiante est calculée pendant le baking et stockée dans le lightmap.
- **SSAO** : Activé via Environment > SSAO dans les WorldEnvironment. Voir Environment SSAO.

## 6. Pourquoi l'AO change tout visuellement



Fig. 19. – Exemple d'Ambient Occlusion baked dans Blender (Cycles, Bake Type: Ambient Occlusion).

Les zones claires reçoivent la lumière ambiante, les zones sombres sont occluses par la géométrie voisine. Remarquez comment les coins, le contact des boîtes avec le sol, et les crevasses sont assombriss — c'est ce qui donne le sens du contact et de la profondeur.

### 💡 L'effet Wadler

Sans AO, une scène 3D paraît « plate » — les coins sont aussi clairs que le centre des murs, les objets semblent flotter. Avec AO, chaque crevasse s'assombrit légèrement, ce qui donne une perception immédiate de **profondeur** et de **contact**. C'est l'un des effets les plus rentables en termes de qualité perçue par rapport au coût de calcul.

## Partie 4 : Les Light Probes — L'Éclairage pour les Objets Dynamiques

### Le problème des objets en mouvement

Les lightmaps éclairent les surfaces **statiques**. Mais un personnage qui marche dans une scène baked n'a pas de lightmap — il est éclairé par des lumières temps réel, ce qui le détache visuellement de l'environnement. Comment lui donner l'éclairage de la scène sans tout recalculer ?

### Light Probes

Les **light probes** sont des points placés dans la scène qui **échantillonnent** l'éclairage baked à leur position. Au runtime, un objet dynamique (personnage, véhicule) interpole l'éclairage entre les probes les plus proches. L'objet reçoit un éclairage qui correspond à son environnement — sans aucun calcul de lumière en temps réel.

- **Placement** : Les probes sont disposées manuellement ou automatiquement dans la scène, surtout autour des zones où des objets dynamiques passent.
- **Stockage** : Chaque probe stocke l'éclairage incoming sous forme d'harmoniques sphériques (SH) — une compression de l'éclairage en 9 coefficients par canal RGB.
- **Interpolation** : Le GPU interpole entre les 4 probes les plus proches (interpolation tétraédrique) pour obtenir l'éclairage à la position de l'objet.

### 💡 Dans Godot

Godot utilise un noeud `LightmapProbe` placé dans la scène. Les objets dynamiques dont le `GI Mode` est réglé sur `Dynamic` utilisent automatiquement les probes pour recevoir l'éclairage baked. Voir `LightmapProbe Class`.

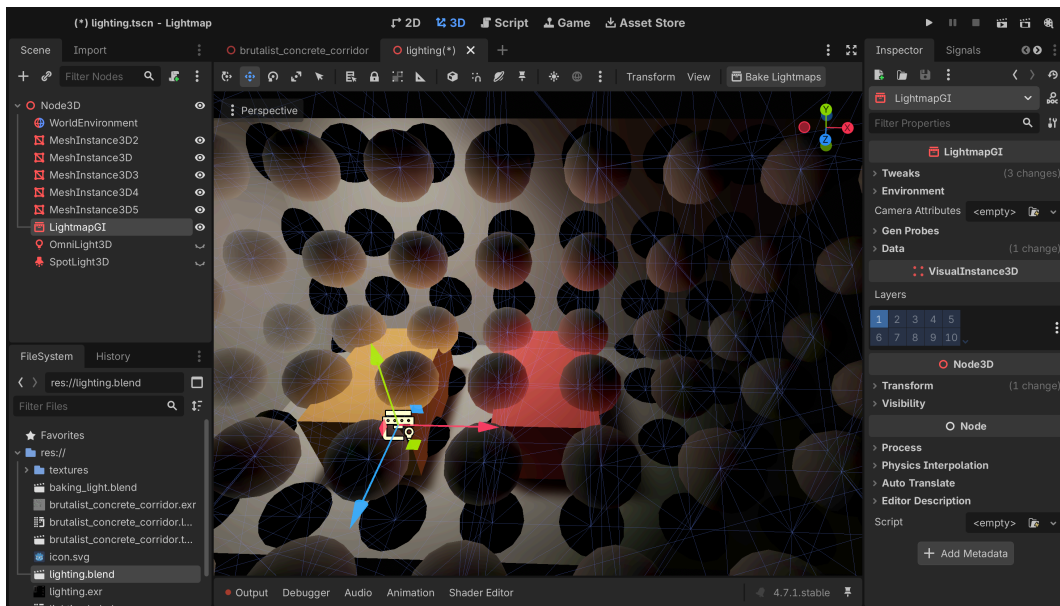


Fig. 20. – Exemple de placement de light probes dans une scène. Les sphères représentent les probes — chacune capture l'éclairage baked à sa position. Un objet dynamique qui se déplace interpole l'éclairage entre les probes les plus proches, ce qui lui donne un éclairage cohérent avec son environnement sans aucun calcul de lumière temps réel.

## 7. Spherical Harmonics — La compression magique

### Pourquoi pas simplement stocker RGB ?

L'éclairage en un point dépend de la **direction** — une surface pointant vers le haut reçoit la lumière du ciel, une surface pointant vers le sol reçoit la lumière réfléchi du sol. Stocker l'éclairage pour toutes les directions nécessiterait un cube map par probe — beaucoup trop de mémoire.

Les **harmoniques sphériques** (SH) décomposent l'éclairage directionnel en une somme de fonctions de base. Avec seulement 9 coefficients par couleur (27 valeurs au total), on peut reconstruire un éclairage **directionnel** de basse fréquence — suffisant pour l'éclairage diffus d'un personnage. C'est une compression avec un ratio d'environ 100:1 par rapport à un cube map.

*Utilisation des SH dans un shader (pseudo-code).*

```
// 9 coefficients par canal (27 valeurs au total)
uniform vec3 shCoeffs[9];

// N = normale du fragment
vec3 evalSH(vec3 N) {
    // Basis functions d'ordre 0, 1, 2
    float sh[9];
    sh[0] = 0.282095; // Y_0^0
    sh[1] = 0.488603 * N.y; // Y_1^-1
    sh[2] = 0.488603 * N.z; // Y_1^0
    sh[3] = 0.488603 * N.x; // Y_1^1
    sh[4] = 1.092548 * N.x * N.y; // Y_2^-2
    sh[5] = 1.092548 * N.y * N.z; // Y_2^-1
    sh[6] = 0.315392 * (3.0 * N.z * N.z - 1.0); // Y_2^0
    sh[7] = 1.092548 * N.x * N.z; // Y_2^1
```

```
sh[8] = 0.546274 * (N.x*N.x - N.y*N.y); // Y_2^2

vec3 color = vec3(0.0);
for (int i = 0; i < 9; i++)
    color += shCoeffs[i] * sh[i];
return color;
}
```

## Partie 5 : L'Éclairage Mixte (Mixed Lighting)

### Le meilleur des deux mondes

L'éclairage **mixte** combine l'éclairage baked (statique) et dynamique (temps réel) :

- **Lumières statiques** (soleil, lampes fixes) : Baked dans les lightmaps.
- **Lumières dynamiques** (lampe torche du joueur, explosions) : Calculées en temps réel.
- **Objets statiques** (murs, sol) : Éclairés par lightmap.
- **Objets dynamiques** (personnages) : Éclairés par light probes + lumières temps réel.

C'est l'approche utilisée par la majorité des jeux commerciaux.

### 8. Les modes d'éclairage mixte dans Godot

| GI Mode  | Statique                 | Dynamique                         | Coût runtime |
|----------|--------------------------|-----------------------------------|--------------|
| Disabled | Aucun                    | Tout temps réel                   | Élevé        |
| Static   | Lightmap +<br>LightmapGI | LightmapProbe<br>pour objets dyn. | Faible       |
| Dynamic  | SDFGI +<br>LightmapProbe | SDFGI temps réel                  | Moyen        |

Tableau 1. – Les trois modes GI de Godot. Static est l'équivalent du baked — le plus utilisé en production. Dynamic utilise SDFGI (Signed Distance Field GI) pour l'illumination globale temps réel.

### Documentation Godot

- GI Modes : MeshInstance3D GI Mode
- SDFGI : SDFGI in Environment
- Global Illumination : Global Illumination Guide

## Partie 6 : Reflection Probes — Les Réflexions Baked

### Le problème des réflexions sur objets baked

Un objet avec un matériau métallique ou brillant réfléchit son environnement. En temps réel, les cube maps dynamiques sont trop chers. Solution : précalculer des cube maps à des positions clés de la scène — les **reflection probes**.

## Reflection Probes

Une **reflection probe** est un cube map de l'environnement rendu (baked) à une position donnée. Au runtime, un objet brillant utilise la probe la plus proche pour calculer sa réflexion. Comme les light probes, on interpole entre plusieurs probes pour les transitions.

- **Baked** : Précalculée hors ligne. Statique mais gratuite.
- **Realtime** : Mise à jour à l'exécution. Coûteuse mais dynamique.
- **Custom** : Cube map fourni manuellement (ex: HDR environment).

## 💡 Dans Godot

Godot utilise le noeud `ReflectionProbe` avec la propriété `update_mode` réglée sur `Once` (baked) ou `Always` (temps réel). Voir `ReflectionProbe Class`.

## Partie 7 : Le Pipeline Complet d'Éclairage Statique

### Workflow type dans un moteur moderne

1. **Préparer la scène** : Placer les lumières statiques, les meshes statiques.
2. **Générer les UV2** : Dépliage automatique ou manuel pour les lightmaps.
3. **Placer les light probes** : Autour des zones de passage des objets dynamiques.
4. **Placer les reflection probes** : Dans les zones avec des matériaux brillants.
5. **Bake** : Lancer le précalcul (secondes à heures selon la scène).
6. **Runtime** : Les objets statiques utilisent les lightmaps, les objets dynamiques utilisent les light probes, les matériaux brillants utilisent les reflection probes.

## Partie 8 : Le Texture Baking pour Personnages

### Pas seulement les scènes !

Le baking ne sert pas qu'à précalculer l'éclairage des murs et du sol. Il est aussi **indispensable** pour les personnages et objets : on bake des détails d'un mesh **high-poly** vers un mesh **low-poly** pour conserver l'apparence sans le coût géométrique. C'est le workflow standard de tout personnage de jeu vidéo.

## 9. Le problème : high-poly vs low-poly

### Le dilemme du modélisateur

Un personnage sculpté dans ZBrush ou Blender peut avoir 10 millions de polygones — beaucoup trop pour un moteur temps réel. On crée donc une version **low-poly** (5 000 à 50 000 triangles) qui sera utilisée au runtime. Mais la version low-poly perd tous les détails fins : cicatrices, écailles, coutures, rugosité.

**Solution** : On **bake** les détails de la high-poly vers des textures appliquées sur la low-poly. Le moteur lit ces textures au runtime — les détails réapparaissent sans le coût géométrique.

| Étape      | Mesh                   | Résultat             |
|------------|------------------------|----------------------|
| 1. Sculpt  | High-poly (10M+ tris)  | Détails géométriques |
| 2. Retopo  | Low-poly (5K-50K tris) | Géométrie légère     |
| 3. Bake    | High → Low             | Textures de détails  |
| 4. Runtime | Low-poly + textures    | Apparence high-poly  |

Tableau 2. – Le workflow high-poly → low-poly : on sculpte les détails, on crée une topologie légère, puis on bake les détails dans des textures.

## 10. Les textures que l'on bake

### Les maps essentielles pour un personnage

- **Normal Map** : La plus importante. Encode les détails géométriques (crevasses, bosses, écailles) sous forme de normales perturbées. Le fragment shader les utilise pour simuler du relief sans géométrie supplémentaire. Vue en détail en Session 7.
- **Ambient Occlusion Map** : Assombrit les crevasses du personnage (aisselle, plis, sous les cheveux). Précalculée depuis la géométrie high-poly, elle ajoute du contact visuel même sans éclairage dynamique.
- **Curvature Map** : Détecte les zones convexes (bords, arêtes) et concaves (creux). Utilisée pour l'usure (poussière dans les creux, éclats sur les arêtes) ou le masquage de matériaux.
- **Cavity Map** : Variante d'AO plus locale — ne capte que les crevasses très fines. Souvent mélangée avec l'AO pour un résultat plus riche.
- **Diffuse/Albedo (baked)** : Parfois on bake la couleur depuis le high-poly (ex: polypainting de ZBrush) vers la low-poly. Utile quand le sculpt contient déjà les couleurs.
- **Position Map / ID Map** : Stocke la position ou un identifiant par région du mesh. Sert pour masquer des zones dans le shader (ex: peau vs métal sur un cyborg).

## 11. Le baking dans Blender

### Le moteur Cycles pour le baking

Blender utilise le moteur **Cycles** pour baker des textures. Le processus :

1. **Préparer les UVs** de la low-poly (UV1, non-chevauchants).
2. **Créer une image texture** vide (ex: 2048×2048) dans l'UV Editor.
3. **Sélectionner** d'abord la high-poly, puis la low-poly (ordre important).
4. **Dans le Render Properties** : sélectionner **Cycles**, puis **Bake**.
5. **Bake Type** : choisir le type de map (Normal, Ambient Occlusion, Diffuse, etc.).
6. **Cage** : Blender projette les rayons depuis la low-poly vers la high-poly. Le **cage** définit la distance maximale de projection — trop petit, des détails sont manqués ; trop grand, des artefacts apparaissent.

*Bake d'une Normal Map dans Blender (workflow).*

1. Importer high-poly et low-poly dans la même scène
2. Sélectionner high-poly → Shift+click low-poly
3. Render Properties > Engine: Cycles
4. Bake > Bake Type: Normal
5. Influence > Space: Tangent (standard pour jeux)
6. Margin: 16px (évite les seams aux bords d'UV)
7. Click "Bake" → l'image se remplit
8. Sauvegarder l'image (PNG ou TGA)

### 💡 Le margin (dilatation)

Les UV islands ne sont jamais parfaitement alignés — il y a des espaces entre eux. Sans **margin**, des pixels noirs apparaissent aux coutures. Le margin (ex: 16px) **dilate** la couleur des islands vers l'extérieur pour remplir ces espaces. C'est un réglage critique pour éviter les seams visibles sur le personnage.

## 12. Utiliser les textures baked dans un moteur

### Assemblage dans le matériau

Les textures baked sont combinées dans le matériau du personnage :

- **Normal Map** → branchée sur le Normal du matériau (via un Normal Map node).
- **AO Map** → multipliée avec l'albedo ou ajoutée comme masque d'assombrissement.
- **Curvature Map** → utilisée comme masque pour l'usure, la poussière, ou les effets de bord.

Au runtime, le personnage low-poly + ses textures baked donne l'illusion d'un mesh high-poly — pour une fraction du coût GPU.

### 💡 Dans Godot

- **Normal Map** : propriété `normal_texture` du `StandardMaterial3D`.
- **AO Map** : propriété `ao_texture` du `StandardMaterial3D`.
- **Roughness/Metallic** : peuvent aussi être baked et branchés sur `roughness_texture` et `metallic_texture`.
- Voir `StandardMaterial3D` Documentation.

## Concepts clés à retenir

### 💡 Les 7 points essentiels

1. **Lightmap** = **éclairage précalculé dans une texture** : Le GPU lit au lieu de calculer. Qualité ray tracing pour le prix d'une texture lookup.
2. **UV2** = **canal UV dédié au lightmap** : Pas de chevauchement, densité uniforme. Les UV1 restent pour les textures de matériau.
3. **Light Probes** = **éclairage pour les objets dynamiques** : Échantillonnent l'éclairage baked à des points clés, interpolé via harmoniques sphériques. Les personnages ne sont plus détachés de la scène.

4. **Ambient Occlusion = assombrissement des crevasses** : Baked (parfait, statique) ou SSAO (dynamique, approximatif). L'effet le plus rentable en qualité perçue.
5. **Éclairage mixte = baked + temps réel** : Les lumières statiques sont baked, les lumières dynamiques (joueur, explosions) restent temps réel. C'est ce que font les jeux commerciaux.
6. **Reflection probes = réflexions baked** : Cube maps précalculés pour les matériaux brillants. Comme les light probes mais pour les réflexions.
7. **Texture baking pour personnages** : On bake les détails du high-poly (normal map, AO, courbure) vers la low-poly. Le personnage léger a l'apparence du sculpt détaillé — c'est le workflow standard de tout jeu commercial.

### 🔗 Question de réflexion pour la prochaine session

« Nous avons vu comment **baker** des normal maps et des AO maps pour les personnages. Mais comment ces textures sont-elles **appliquées** sur une surface ? Comment répéter une texture de brique sur un mur infini ? Comment éviter le flou quand on regarde une texture de loin ? Et quels formats de texture utilise-t-on en production ? »

**Réponse** : C'est la **texturisation de base**, que nous verrons en Session 7 — UV1, tiling, mipmaps, formats de compression, et sampler filtering.

## Travail Pratique : Baked Lighting en Three.js

### 🔗 Objectif du TP

Mettre en pratique les concepts de lightmapping et d'environnement map. Vous allez :

1. Générer une scène dans Blender avec un script Python
2. Baker l'éclairage dans Blender (Cycles)
3. Exporter la scène en .glb
4. Appliquer le lightmap et un environment map en Three.js

## Étape 1 : Générer la scène dans Blender

### Script Python fourni

Un script `generate_bake_scene.py` est fourni. Il crée :

- Une pièce (sol, 3 murs, plafond)
- 3 boîtes de tailles différentes
- 2 lumières (soleil + ampoule)
- Les UV2 dépliés automatiquement (prêts pour le baking)

### Procédure :

1. Ouvrez Blender > onglet **Scripting**
2. Collez le contenu de `generate_bake_scene.py`
3. Exécutez le script (bouton **Run Script**)
4. La scène apparaît avec les objets, lumières et UV2

## Étape 2 : Baker le lightmap dans Blender

### Baking avec Cycles

1. Dans **Render Properties** : moteur **Cycles**, **GPU** si disponible
2. Créez une nouvelle image dans l'UV Editor (2048×2048, 32-bit Float si EXR)
3. Sélectionnez tous les meshes (Shift+A > Select All)
4. **Bake > Bake Type : Combined**
5. **Margin** : 16px
6. Cliquez **Bake** — l'image se remplit
7. Sauvegardez l'image : session06\_lightmap.png (ou .exr pour HDR)

### 💡 Conseils

- **Combined** bake inclut diffus + ombres + AO en une seule texture
- Si le résultat est trop sombre, augmentez l'énergie des lumières
- Vérifiez que les UV2 sont bien actifs avant de baker

## Étape 3 : Exporter en .glb

### Export glTF

1. **File > Export > glTF 2.0 (.glb)**
2. Options : cochez **UVs**, décochez **Compression** pour simplifier
3. Nommez le fichier session06\_bake\_scene.glb
4. Placez-le dans le même dossier que les fichiers Three.js

## Étape 4 : Appliquer le lightmap en Three.js

### Le fichier de départ

Un fichier session06\_baked\_lighting.js est fourni avec :

- Le chargement de la scène .glb via GLTFLoader
- Une GUI pour comparer lightmap vs lumières temps réel
- Des **TODO ÉTUDIANT** marqués dans le code

### Votre mission :

1. Charger la texture de lightmap (session06\_lightmap.png)
2. L'assigner à material.lightMap sur chaque mesh
3. Régler lightMapIntensity
4. (Bonus) Charger un environment map HDR avec RGBELoader
5. (Bonus) Comparer les FPS : lightmap vs lumières temps réel

### ⚠️ Pièges fréquents

- **Color space** : Le lightmap doit être en sRGBColorSpace si bake en sRGB, ou linéaire si EXR
- **UV2** : Three.js utilise le deuxième canal UV automatiquement pour lightMap — vérifiez que le .glb contient bien les UV2

- **glTF export** : Assurez-vous que les UV2 sont exportés (option **UVs** cochée)
- **Intensité** : Si la scène est trop sombre/claire, ajustez `lightMapIntensity` (0 à 3)

### 💡 Résultat attendu

À la fin du TP, vous devriez avoir une scène 3D éclairée **sans aucune lumière temps réel** — tout vient du lightmap. La GUI permet de :

- Activer/désactiver le lightmap
- Activer/désactiver l'environnement map
- Activer/désactiver les lumières temps réel pour comparer

La différence de FPS entre lightmap et lumières temps réel devrait être significative — c'est tout l'intérêt du baking !

## Session 7 : Création de Post-Process dans Babylon.js

### 🔗 Objectifs de la session

Apprendre à appliquer des effets visuels après le rendu de la scène (post-processing) : flou, bloom, correction de couleur, etc.

## Qu'est-ce qu'un Post-Process ?

### Définition

Un post-process est un effet appliqué sur l'image finale rendue (framebuffer) avant son affichage. Il s'exécute dans un shader plein écran (fullscreen quad) qui lit la texture de rendu et applique une transformation par pixel.

## Post-Process Pipeline dans Babylon.js

*Pipeline de post-process.*

```
const pipeline = new BABYLON.DefaultRenderingPipeline(
  "default", true, scene, [camera]
);
pipeline.bloomEnabled = true;
pipeline.bloomThreshold = 0.8;
pipeline.bloomWeight = 0.3;

pipeline.imageProcessing.contrast = 1.2;
pipeline.imageProcessing.exposure = 1.0;
pipeline.imageProcessing.vignetteEnabled = true;
```

## Post-Process Intégrés

### Effets disponibles

- **Bloom** : Halos lumineux autour des zones brillantes.
- **Depth of Field (DoF)** : Flou de profondeur (arrière-plan flou).
- **Vignette** : Assombrissement des bords de l'image.
- **Chromatic Aberration** : Décalage des canaux RGB (effet lentille).
- **Grain** : Bruit filmique.
- **Fxaa** : Anti-aliasing rapide.
- **Screen Space Reflections (SSR)** : Réflections temps réel.

## Post-Process Custom

*Shader personnalisé en post-process.*

```
const postProcess = new BABYLON.PostProcess(
  "custom", "customEffect", ["time"], null, 1.0, camera
);
```

```
BABYLON.Effect.ShadersStore["customEffectFragmentShader"] = `
  varying vec2 vUV;
```

```

uniform sampler2D textureSampler;
uniform float time;

void main() {
    vec2 uv = vUV;
    uv.x += sin(uv.y * 10.0 + time) * 0.01;
    gl_FragColor = texture2D(textureSampler, uv);
}
`;

postProcess.onApply = (effect) => {
    effect.setFloat("time", performance.now() / 1000);
};
    
```

## HDR et Tone Mapping

### Tone Mapping

Le rendu HDR (High Dynamic Range) calcule la lumière avec des valeurs supérieures à 1.0. Le tone mapping ramène ces valeurs dans la plage  $[0, 1]$  pour l’affichage. Babylon.js propose plusieurs opérateurs : ACES, Reinhard, etc.

## Session 8 : Rendering Pipeline (Partie 1)

### 🔗 Objectifs de la session

Comprendre l'architecture du pipeline de rendu : du modèle 3D au pixel à l'écran.

## Vue d'Ensemble du Pipeline

### Étapes du pipeline graphique

1. **Application Stage (CPU)** : Le code (JS) prépare les données : matrices, paramètres, culling.
2. **Geometry Processing (GPU)** : Vertex Shader, Tessellation, Geometry Shader.
3. **Rasterization (GPU)** : Conversion des triangles en fragments (pixels).
4. **Pixel Processing (GPU)** : Fragment Shader, tests de profondeur, blending.
5. **Output Merger** : Écriture dans le framebuffer.

## Vertex Shader en Détail

### Responsabilités

Le Vertex Shader transforme chaque sommet :

- Object Space → World Space (matrice **world**)
- World Space → View Space (matrice **view**)
- View Space → Clip Space (matrice **projection**)
- $$MVP = P \times V \times W$$

Il peut aussi passer des données au Fragment Shader (normales, UVs, couleurs).

## Rasterization

### Du triangle au pixel

Le rasterizer détermine quels pixels sont couverts par chaque triangle. Il interpole les varyings (UV, normales, couleurs) à travers la surface du triangle.

## Fragment Shader en Détail

### Responsabilités

Le Fragment Shader calcule la couleur finale de chaque pixel :

- Sampling de textures
- Calculs d'éclairage
- Effets procéduraux

Il ne connaît rien de la géométrie — seulement des varyings interpolés.

## Tests et Blending

### Tests du framebuffer

- **Depth Test** : Compare la profondeur du fragment avec le Z-buffer. Si le fragment est derrière, il est ignoré.
- **Stencil Test** : Masque basé sur un buffer de stencil (ex: silhouettes, reflets).
- **Blending** : Mélange la couleur du fragment avec celle du framebuffer (transparence).

## Session 9 : Rendering Pipeline (Partie 2)

### 🔗 Objectifs de la session

Approfondir le pipeline : techniques avancées de rendu, deferred rendering, et architecture moderne.

## Forward vs Deferred Rendering

### Forward Rendering

Chaque objet est rendu un par un. Pour chaque fragment, on calcule l'éclairage de toutes les lumières.

- **Avantage** : Simple, MSAA compatible, bon pour peu de lumières.
- **Inconvénient** : Coût  $O(\text{objets} \times \text{lumières})$ , overdraw (pixels rendus plusieurs fois).

### Deferred Rendering

On rend d'abord la géométrie dans des G-Buffers (positions, normales, couleurs, profondeur). L'éclairage est calculé dans un second pass en plein écran.

- **Avantage** : Coût d'éclairage indépendant du nombre d'objets,  $O(\text{pixels} \times \text{lumières})$ .
- **Inconvénient** : Gourmand en mémoire (G-Buffers), MSAA difficile, transparence problématique.

## G-Buffer

### Composition typique

- **RTo** : Albedo (RGB) + AO (A)
- **RT1** : Normales (RGB) + Roughness (A)
- **RT2** : Metallic (R) + Specular (G) + Depth (B)

Chaque Render Target stocke des informations géométriques et matérielles.

## Culling

### Optimisations de géométrie

- **Frustum Culling** : On ignore les objets hors du champ de vision de la caméra.
- **Occlusion Culling** : On ignore les objets cachés derrière d'autres.
- **Backface Culling** : On ignore les faces arrière des triangles (normale pointant away).

## Draw Calls et Batching

### 💡 Le goulot d'étranglement

Chaque **draw call** (appel de rendu) a un coût CPU. Réduire le nombre de draw calls est crucial :

- **Instancing** : Rendre N copies d'un même mesh en un seul draw call.

- **Merging** : Fusionner les meshes statiques.
- **Texture Atlas** : Regrouper les textures pour éviter de changer de matériau.

## Babylon.js et le Pipeline

### Rendering Pipeline de Babylon.js

Babylon.js utilise par défaut le Forward Rendering. Le Deferred Rendering est disponible via le plugin `@babylonjs/materials`. La scène gère automatiquement le frustum culling, le tri des objets transparents, et le batching des lumières.

# Session 10 : Techniques d'Optimisation et Profilage (Partie 1)

## 🔗 Objectifs de la session

Apprendre à mesurer et optimiser les performances d'une application 3D temps réel.

## Pourquoi Optimiser ?

### Le budget de performance

Pour maintenir 60 FPS, chaque frame doit être rendue en moins de 16.6ms. Le rendu graphique n'est qu'une partie de ce budget (avec la logique de jeu, l'audio, le réseau).

## Outils de Profilage

### Outils disponibles

- **Babylon.js Inspector** : `scene.debugLayer.show()` — statistiques de scène, draw calls, FPS, mémoire GPU.
- **Chrome DevTools** : Performance tab, GPU tab, Memory tab.
- **WebGL Inspector / Spector.js** : Capture des draw calls WebGL, inspection des buffers et shaders.

## Métriques Clés

### Indicateurs de performance

- **FPS** : Images par seconde (cible: 60).
- **Draw Calls** : Nombre d'appels de rendu par frame (cible: < 1000).
- **Triangles** : Nombre total de triangles rendus.
- **Texture Memory** : Mémoire GPU utilisée par les textures.
- **Shader Complexity** : Nombre d'instructions dans les shaders.

## Optimisation Géométrique

### Réduction de la complexité

- **LOD (Level of Detail)** : Plusieurs versions d'un mesh avec des niveaux de détail décroissants. On choisit le LOD selon la distance à la caméra.
- **Mesh Simplification** : Réduction du nombre de polygones.
- **Frustum Culling** : Déjà géré par Babylon.js, mais peut être amélioré avec des **occluders**.

## Optimisation des Draw Calls

### Techniques de batching

- **Thin Instances** : Babylon.js permet de rendre des milliers d'instances d'un mesh en un seul draw call.

```
mesh.thinInstanceCount = 1000;  
mesh.thinInstanceSetBuffer("matrix", matricesBuffer, 16);
```

- **Merge Meshes** : BABYLON.Mesh.MergeMeshes(meshes) fusionne plusieurs meshes statiques en un seul.

# Session 11 : Techniques d'Optimisation et Profilage (Partie 2)

## 🔗 Objectifs de la session

Optimisation des textures, des shaders, et techniques avancées de performance GPU.

## Optimisation des Textures

### Stratégies

- **Texture Compression** : Formats compressés GPU (BasisU, KTX2) qui réduisent la mémoire et la bande passante.
- **Mipmapping** : Versions pré-calculées de la texture à différentes résolutions. Le GPU choisit la mip level selon la distance.
- **Texture Atlas** : Regrouper plusieurs textures en une seule pour réduire les changements de matériau.
- **Resolution** : Adapter la résolution des textures à l'importance visuelle de l'objet.

## Optimisation des Shaders

### Bonnes pratiques GLSL

- Éviter les branches conditionnelles (`if`) dans les shaders — préférer `mix`, `step`.
- Minimiser les calculs dans le Fragment Shader (il s'exécute millions de fois par frame).
- Précalculer ce qui peut l'être dans le Vertex Shader.
- Utiliser les built-in GLSL (plus optimisés que les fonctions custom).

## Level of Detail (LOD) dans Babylon.js

*LOD automatique.*

```
mesh.addLODLevel(20, highDetailMesh);  
mesh.addLODLevel(50, mediumDetailMesh);  
mesh.addLODLevel(100, lowDetailMesh);  
mesh.addLODLevel(200, null); // Rien rendu au-delà
```

## Gestion de la Mémoire GPU

### 💡 Règles d'or

- Libérer les ressources : `mesh.dispose()`, `texture.dispose()`, `material.dispose()`.
- Réutiliser les géométries (un seul mesh, multiples instances).
- Surveiller les fuites : le Babylon.js Inspector affiche la mémoire GPU.

## WebGPU vs WebGL

### Le futur du rendu web

Babylon.js supporte WebGPU, la nouvelle API graphique du web. WebGPU offre :

- Accès plus direct au GPU (moins d'overhead CPU).
- Compute Shaders (calculs généraux sur GPU).
- Meilleure performance pour les scènes complexes.

On bascule entre WebGL et WebGPU via :

```
const engine = new BABYLON.WebGPUEngine(canvas);  
await engine.initAsync();
```

# Session 12 : Exploration d'une API de Programmation Graphique

## 🔗 Objectifs de la session

Plonger dans une API graphique bas niveau (WebGPU) pour comprendre ce qui se cache sous l'abstraction de Babylon.js.

## APIs Graphiques : Vue d'Ensemble

### Niveaux d'abstraction

- **Haut niveau** : Babylon.js, Three.js — moteur complet, scène, matériaux, éclairage.
- **Moyen niveau** : WebGL, WebGPU — accès direct au pipeline GPU, mais gestion manuelle.
- **Bas niveau** : OpenGL, Vulkan, DirectX 12, Metal — APIs natives, contrôle total.

## WebGPU : La Nouvelle API Web

### Concepts fondamentaux

- **Adapter** : Représente le GPU physique.
- **Device** : La connexion logique au GPU.
- **Pipeline** : Décrit le rendu (shaders, format de sortie, blend mode).
- **Buffers** : Données envoyées au GPU (vertex, index, uniform).
- **Textures** : Images stockées sur le GPU.
- **Command Encoder** : Enregistre les commandes de rendu, soumis au GPU en un seul **submit**.

## Comparaison WebGL vs WebGPU

### Différences clés

| Aspect          | WebGL       | WebGPU                       |
|-----------------|-------------|------------------------------|
| Shader Language | GLSL        | WGSL                         |
| Paradigme       | État global | Objets explicites            |
| Compute         | Non         | Oui (Compute Shaders)        |
| Performance     | Bonne       | Meilleure (moins d'overhead) |
| Maturité        | Très mature | Récent, en évolution         |

## WebGPU dans Babylon.js

*Engine WebGPU.*

```
const engine = new BABYLON.WebGPUEngine(canvas);
await engine.initAsync();
```

```
const scene = new BABYLON.Scene(engine);
// ... même code que WebGL
```

```
engine.runRenderLoop(() => scene.render());
```

Babylon.js abstrait la différence entre WebGL et WebGPU. Le même code de scène fonctionne sur les deux.

Pour les compute shaders :

```
const computeShader = new BABYLON.ComputeShader(
    "compute", engine, { computeSource: wgsCode },
    [{ type: "storage", name: "output", buffer: storageBuffer }]
);
computeShader.dispatch(64, 1, 1);
```

## WGSL : Le Langage de Shader WebGPU

### Exemple WGSL

```
@vertex
fn vs_main(@location(0) position: vec3<f32>) -> @builtin(position) vec4<f32> {
    return vec4<f32>(position, 1.0);
}

@fragment
fn fs_main() -> @location(0) vec4<f32> {
    return vec4<f32>(1.0, 0.0, 0.0, 1.0);
}
```

WGSL est plus typé et structuré que GLSL, conçu pour la sécurité et la performance.

## Conclusion du Cours

### 💡 Récapitulatif

Au cours de cette session, nous avons exploré l'API sous-jacente qui alimente le rendu moderne sur le web. Comprendre WebGPU permet de mieux utiliser Babylon.js et d'optimiser les performances au maximum.